

Relation Algebra

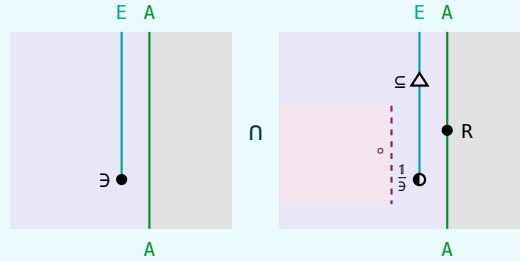
§13. Optimisation Problems

§13.1. $\text{est}(R) \triangleq \exists n(\epsilon \setminus R^\circ)$

(13.1a)

For $R : A \rightarrow A$, $\text{est}(R) \triangleq \exists n(\epsilon \setminus R^\circ) : EA \rightarrow A$ $X \in \epsilon \setminus R^\circ \iff EX \in R^\circ$, and EX runs $y \rightarrow XS \rightarrow X$, member first, so its pair (y, x) is in R° exactly when $x R y$; at $R \triangleq \leq$ that is $x \leq y$ for every $y \in xs$, the least — $\epsilon \setminus \leq$ would give $y \leq x$, the greatest.

$\text{est}(R) = \exists n \subseteq (\frac{R}{\exists})^\circ$ x is the $\text{est}(R)$ of xs when $x \in xs$ and $xs \subseteq \Lambda(R)(x)$, the set of every y with $x R y$ — mem-ldiv at $Z \triangleq R^\circ$



$xs (\text{est}(R)) x \iff x \in xs \wedge (\forall y \in xs. x R y)$ the same predicate under the same letter, $\min R$, so $\text{est}(R) = \min(R^\circ)$ once R is an arrow;

$xs (\text{est}(R)) x \iff (x \text{ in } xs) \text{ and all } x R \setminus : xs \text{ in } q$

$\text{est}(R) = \exists n \text{ all } R^\circ$ all $R \triangleq \epsilon \setminus R$, q 's all; the chains below keep it written $\epsilon \setminus$

$E(R) \triangleq \frac{\exists R}{\exists} : EA \rightarrow EB$, $xs E(R) ys \iff ys = \{y \mid \exists x \in xs. x R y\}$ the image of xs , pow-laws

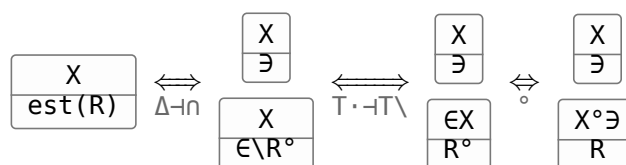
$P(R) : EA \rightarrow EB$, $xs P(R) ys \iff (\forall x \in xs. \exists y \in ys. x R y) \wedge (\forall y \in ys. \exists x \in xs. x R y)$ every x and every y has a partner, powrel-readings

(13.1b)

the law	what it says
$X \in \text{est}(R) \iff X \in \exists$ and $X^\circ \in R$	in the set, and below every element of it
$\frac{1}{\exists} (\epsilon \setminus R) = R$	bounding a singleton is bounding its element
$\frac{S}{\exists} (\epsilon \setminus R) = S^\circ \setminus R$	bound S 's image without building the set
$\text{union} \triangleq \frac{\exists \exists}{\exists} : E(EA) \rightarrow EA$	flattens a set of sets
$\text{union} (\epsilon \setminus R) = \epsilon \setminus (\epsilon \setminus R)$	bound a union by bounding each member set
$\frac{1}{\exists} \text{est}(R) = 1 \cap R^\circ$	a singleton's minimum is its element, where R is reflexive $\frac{S}{\exists} \text{est}(R)$ at $S := 1$
$\frac{S}{\exists} \text{est}(R) = S \cap (S^\circ \setminus R^\circ)$	an S -value that points to every S -value
$\frac{S}{\exists} \text{est}(R \cap S^\circ S) = \frac{S}{\exists} \text{est}(R)$	only R between values S gives one argument counts — context
$E(S) \text{est}(R) = (\exists S) \cap ((\exists S)^\circ \setminus R^\circ)$	the same for the image of a set $\frac{S}{\exists} \text{est}(R)$ at $S := \exists S$
$\text{Map}(f) \implies P(f) \text{est}(R) = \text{est}(f R f^\circ) f$	shunt a function through a minimum
$P(S) \text{est}(R) = (\exists S) \cap (\epsilon \setminus (S R^\circ))$ R reflexive	fusion with the power relator $\exists$ is the only proof here that tabulates
$P(S) \text{est}(R) \subseteq (\exists S) \cap (\epsilon \setminus (S R^\circ))$	the half of the row above that costs nothing
$P(\text{est}(R)) \text{est}(R) \subseteq \text{union est}(R)$ R transitive	a minimum in each set, then a minimum of those
$RR \subseteq R \implies P(\text{est}(R)) \text{est}(R) = P(\text{dom}(\text{est}(R))) \text{union est}(R)$	the same as an equality, once empty sets are dropped

§13.1.1. $X \in \text{est}(R) \iff X \in \exists$ and $X^\circ \in R$

(13.1.1a)



§13.1.2. $\frac{1}{\exists} (\epsilon \setminus R) = R$

(13.1.2a)

$$\begin{array}{c} X \\ \hline \frac{1}{\exists} (\epsilon \backslash R) \end{array} \xleftrightarrow{f^\circ \cdot \neg f^\circ} \begin{array}{c} \frac{1}{\exists}^\circ X \\ \hline \epsilon \backslash R \end{array} \xleftrightarrow{T^\circ \cdot \neg T^\circ} \begin{array}{c} \epsilon \frac{1}{\exists}^\circ X \\ \hline R \end{array} \xleftrightarrow{\circ} \begin{array}{c} (\frac{1}{\exists} \exists)^\circ X \\ \hline R \end{array} \xleftrightarrow{i \cdot \neg E} \begin{array}{c} X \\ \hline R \end{array}$$

§13.1.3. $\frac{S}{\exists} (\epsilon \backslash R) = S^\circ \backslash R$

$\frac{S}{\exists}$ gathers the S -image of a point into one set and $\epsilon \backslash R$ asks that every member of that set be R -related to the target, so the set cancels and $S^\circ \backslash R$ asks it of the S -image directly.

(13.1.3a)

$$\begin{array}{c} X \\ \hline \frac{S}{\exists} (\epsilon \backslash R) \end{array} \xleftrightarrow{f^\circ \cdot \neg f^\circ} \begin{array}{c} \frac{S}{\exists}^\circ X \\ \hline \epsilon \backslash R \end{array} \xleftrightarrow{T^\circ \cdot \neg T^\circ} \begin{array}{c} \epsilon \frac{S}{\exists}^\circ X \\ \hline R \end{array} \xleftrightarrow{\circ} \begin{array}{c} (\frac{S}{\exists} \exists)^\circ X \\ \hline R \end{array} \xleftrightarrow{\cdot \exists \neg \frac{S}{\exists}} \begin{array}{c} S^\circ X \\ \hline R \end{array} \xleftrightarrow{T^\circ \cdot \neg T^\circ} \begin{array}{c} X \\ \hline S^\circ \backslash R \end{array}$$

§13.1.4. $\text{union} (\epsilon \backslash R) = \epsilon \backslash (\text{union} R)$

(13.1.4a)

$$\begin{array}{c} X \\ \hline \text{union} (\epsilon \backslash R) \end{array} \xleftrightarrow{f^\circ \cdot \neg f^\circ} \begin{array}{c} \text{union}^\circ X \\ \hline \epsilon \backslash R \end{array} \xleftrightarrow{T^\circ \cdot \neg T^\circ} \begin{array}{c} \epsilon \text{union}^\circ X \\ \hline R \end{array} \xleftrightarrow{\circ} \begin{array}{c} (\text{union} \exists)^\circ X \\ \hline R \end{array} \xleftrightarrow{\cdot \exists \neg \frac{S}{\exists}} \begin{array}{c} (\exists \exists)^\circ X \\ \hline R \end{array} \xleftrightarrow{\circ, T^\circ \cdot \neg T^\circ} \begin{array}{c} X \\ \hline \epsilon \backslash (\text{union} R) \end{array}$$

§13.1.5. $\frac{S}{\exists} \text{est}(R) = S \cap (S^\circ \backslash R^\circ)$

(13.1.5a)

$$\begin{array}{c} X \\ \hline \frac{S}{\exists} \text{est}(R) \end{array} \xleftrightarrow{f^\circ \cdot \neg f^\circ} \begin{array}{c} \frac{S}{\exists}^\circ X \\ \hline \text{est}(R) \end{array} \xleftrightarrow{\Delta \neg \cap} \begin{array}{c} \frac{S}{\exists}^\circ X \\ \hline \exists \end{array} \xleftrightarrow{f^\circ \cdot \neg f^\circ} \begin{array}{c} X \\ \hline \frac{S}{\exists} \exists \end{array} \xleftrightarrow{\cdot \exists \neg \frac{S}{\exists}, (13.1.3a)} \begin{array}{c} X \\ S \end{array} \xleftrightarrow{\Delta \neg \cap} \begin{array}{c} X \\ \hline S \cap (S^\circ \backslash R^\circ) \end{array}$$

§13.1.6. $\frac{S}{\exists} \text{est}(R) = \frac{S}{\exists} \text{est}(R \cap S^\circ S)$

(13.1.6a)

$$\begin{array}{c} X \\ \hline \frac{S}{\exists} \text{est}(R \cap S^\circ S) \end{array} \xleftrightarrow{(13.1.5a), \circ} \begin{array}{c} X \\ \hline S \cap (S^\circ \backslash (R^\circ \cap S^\circ S)) \end{array} \xleftrightarrow{\Delta \neg \cap, T^\circ \cdot \neg T^\circ} \begin{array}{c} X \\ S \end{array} \xleftrightarrow{\Delta \neg \cap, S^\circ \cdot \text{monotone}} \begin{array}{c} X \\ S \end{array} \xleftrightarrow{T^\circ \cdot \neg T^\circ, \Delta \neg \cap} \begin{array}{c} X \\ \hline S \cap (S^\circ \backslash R^\circ) \end{array} \xleftrightarrow{(13.1.5a)} \begin{array}{c} X \\ \hline \frac{S}{\exists} \text{est}(R) \end{array}$$

§13.1.7. $P(f) \text{est}(R) = \text{est}(f R f^\circ) f$

(13.1.7a)

$$\begin{array}{c} P(f) \text{est}(R) \\ \hline f \text{ a map} \end{array} \xleftrightarrow{P=E \text{ on maps, (13.1.5a)}} \begin{array}{c} (\exists f) \cap ((\exists f)^\circ \backslash R^\circ) \end{array} \xleftrightarrow{\circ, T^\circ \cdot \neg T^\circ, f^\circ \cdot \neg f^\circ} \begin{array}{c} (\exists f) \cap (\epsilon \backslash (f R^\circ)) \end{array} \xleftrightarrow{\text{modular law, } f \text{ simple}} \begin{array}{c} (\exists \cap ((\epsilon \backslash (f R^\circ)) f^\circ)) f \end{array} \xleftrightarrow{\cdot f \neg f^\circ, \circ, \text{est}} \begin{array}{c} \text{est}(f R f^\circ) f \end{array}$$

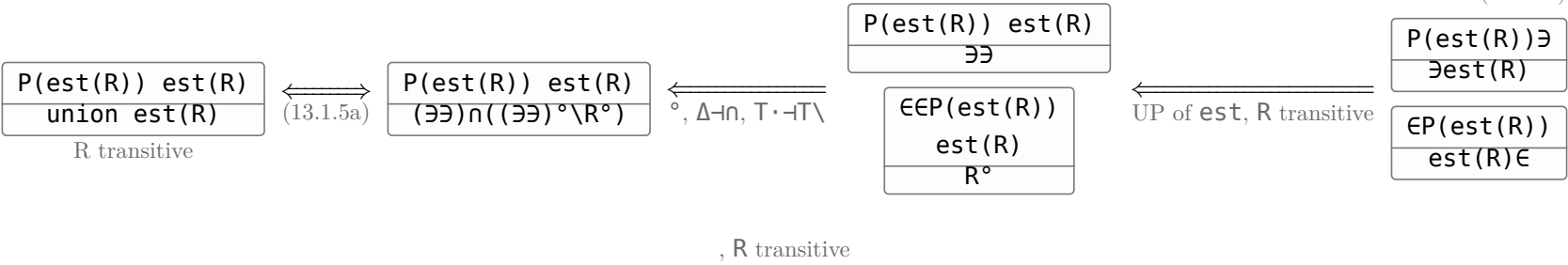
§13.1.8. $P(S) \text{est}(R) \subseteq (\exists S) \cap (\epsilon \backslash (S R^\circ))$

(13.1.8a)

$$\begin{array}{c} P(S) \text{est}(R) \\ \hline (\exists S) \cap (\epsilon \backslash (S R^\circ)) \end{array} \xleftrightarrow{\Delta \neg \cap, T^\circ \cdot \neg T^\circ} \begin{array}{c} P(S) \text{est}(R) \\ \hline \exists S \end{array} \xleftrightarrow{\text{UP of est}} \begin{array}{c} P(S) \exists \\ \hline \exists S \end{array} \xleftrightarrow{\text{UP of est}} \begin{array}{c} \epsilon P(S) \text{est}(R) \\ \hline S R^\circ \end{array} \xleftrightarrow{\text{UP of est}} \begin{array}{c} \epsilon P(S) \\ \hline S \epsilon \end{array}$$

§13.1.9. $P(\text{est}(R)) \text{est}(R) \subseteq \text{union est}(R)$

(13.1.9a)



§13.2. Lax natural transformations (LaT)

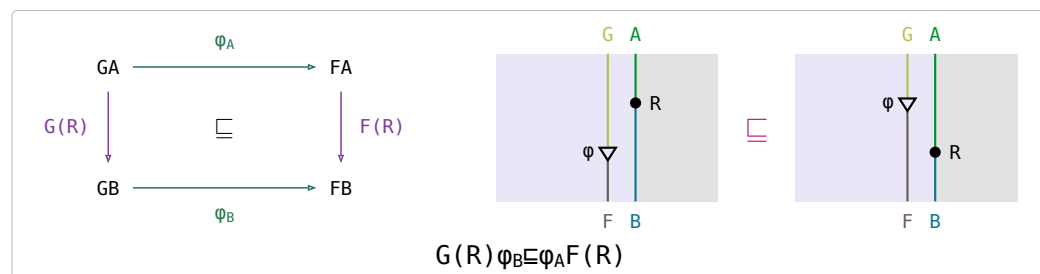
(13.2a)

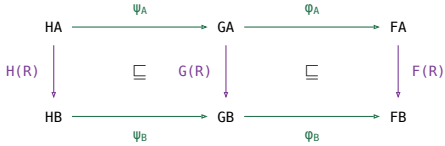
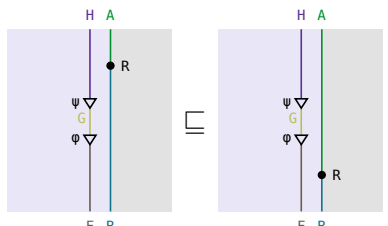
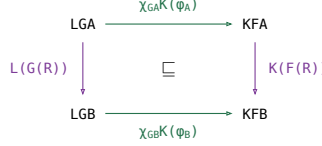
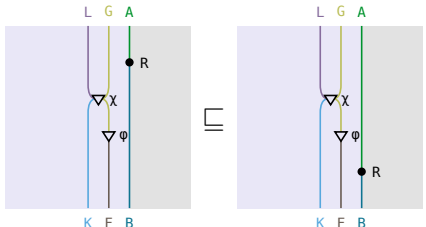
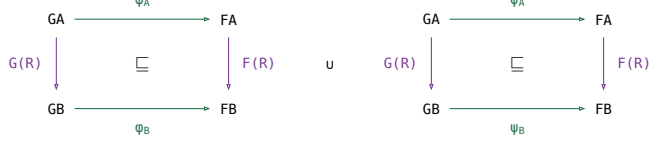
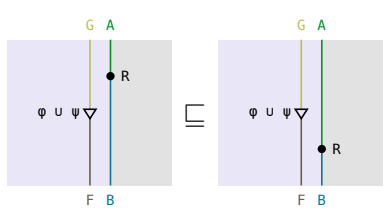
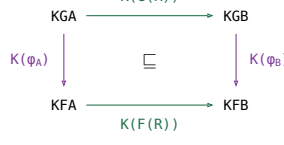
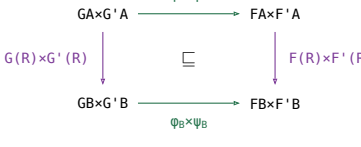
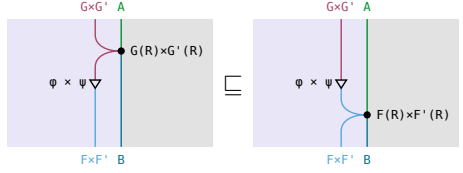
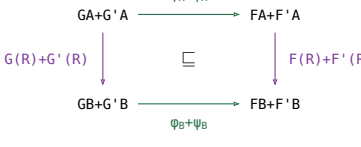
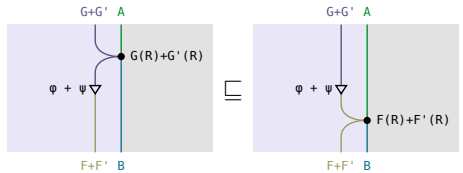
For relators $G, F : \mathcal{C} \rightarrow \mathcal{D}$ and components $\varphi_A : GA \rightarrow FA$, φ is **lax at** $R : A \rightarrow B$ when $G(R)\varphi_B \sqsubseteq \varphi_A F(R)$ — both sides $GA \rightarrow FB$, the left through the component at B , the right through the one at A .

$\varphi : G \Rightarrow F$ is a **lax natural transformation** (LaT) when it is lax at **every** R . (5.13)

Lax at every **map** already gives LaT, and at a map the inequation is an equality $G(f)\varphi = \varphi F(f)$: laxness is about relations only.

(13.2b)



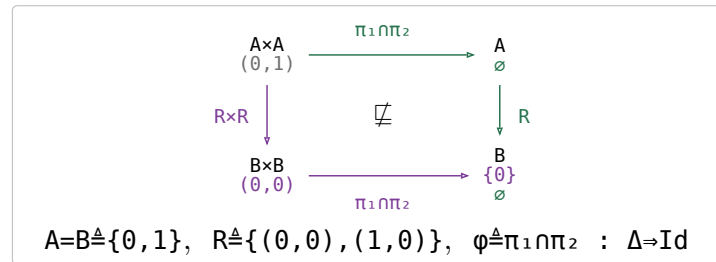
closed under	commutative diagram	string diagram
composition $\psi\phi$	 $H(R)\psi_B \sqsubseteq \psi_A G(R)$ and $G(R)\phi_B \sqsubseteq \phi_A F(R)$ give $H(R)(\psi_B\phi_B) \sqsubseteq (\psi_A\phi_A)F(R)$	
horizontal composition $\chi\phi$	 $\phi : G \Rightarrow F$ with $G, F : \mathcal{C} \rightarrow \mathcal{D}$ and $\chi : L \Rightarrow K$ with $L, K : \mathcal{D} \rightarrow \mathcal{E}$ give $\chi \circ \phi : L \circ G \Rightarrow K \circ F$, the family $A \mapsto \chi_{GA} K(\phi_A)$ $L(G(R))\chi_{GB} \sqsubseteq \chi_{GA} K(G(R))$ is χ lax at $G(R)$; then $K(G(R)\phi_B) \sqsubseteq K(\phi_A F(R))$ is K applied to ϕ's own inequation; $\chi := \mathbb{1}_K$ gives $K(\phi) : K \circ G \Rightarrow K \circ F$, K composed on the outside; $\phi := \mathbb{1}_G$ gives $\chi G : L \circ G \Rightarrow K \circ G$, G composed on the inside two candidates, $\chi_{GA} K(\phi_A)$ and $L(\phi_A)\chi_{FA}$, ordered by $\sqsubseteq$; this row is the first	
union $\phi \cup \psi$	 $G(R)\phi_B \sqsubseteq \phi_A F(R)$ and $G(R)\psi_B \sqsubseteq \psi_A F(R)$ give $G(R)(\phi_B \cup \psi_B) \sqsubseteq (\phi_A \cup \psi_A)F(R)$	
a relator K $K(\phi)$	 $G(R)\phi_B \sqsubseteq \phi_A F(R)$ gives $K(G(R))K(\phi_B) \sqsubseteq K(\phi_A)K(F(R))$	
product $\phi \times \psi$	 $\phi : G \Rightarrow F$ and $\psi : G' \Rightarrow F'$ give $\phi \times \psi : G \times G' \Rightarrow F \times F'$ $(R \times S)(U \times V) = (RU) \times (SV)$ and monotonicity in both slots — the row above's K applied to the inequation, run on a bifunctor; the fork is the derived case $\langle \phi, \psi \rangle = \triangleleft(\phi \times \psi)$, and its ONE cost is the lax copy law $R \triangleleft \triangleleft (R \times R)$ — rel-monoid	
coproduct $\phi + \psi$	 $\phi : G \Rightarrow F$ and $\psi : G' \Rightarrow F'$ give $\phi + \psi : G + G' \Rightarrow F + F'$ $(R + S)(U + V) = (RU) + (SV)$ and monotonicity in both slots; the co-fork is the derived case $[\phi, \psi] = (\phi + \psi) \triangleright$, and $\triangleright$ costs nothing the step it would need is $(\phi_A F(R)) \cap (\psi_A F(R)) \sqsubseteq (\phi \cap \psi)_A F(R)$, the wrong direction of meet-semidistrib	
meet — fails $\phi \cap \psi$	a counterexample: (13.2.1a)	

Each row is the general slide $R_A X \sqsubseteq X' R_B$ and $R_B Y \sqsubseteq Y' R_C$ give $R_A (XY) \sqsubseteq (X' Y') R_C$ at $R_A, R_B := G(R), F(R)$ and $X, X' := \phi_B, \phi_A$, quantified over R ; at $X' = X$ with R_A, R_B endorelations it is the monotonicity reading instead — the $\forall R$ sits outside the law, and is the only difference

§13.2.1. meet is not closed in LaT

counterexample — $\varphi, \psi : \mathbf{G} \Rightarrow \mathbf{F}$ lax natural does NOT give $\varphi \cap \psi$ lax natural

(13.2.1a)



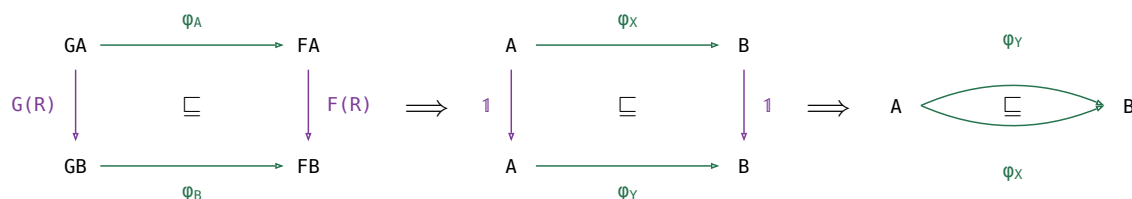
§13.2.2. Two stacked towers

(13.2.2a)

	lower — inside ONE allegory	upper — between allegories
0-cell	an object A	an allegory $\mathcal{C}$ — a whole lower tower
1-cell	a relation $R : A \rightarrow B$	a relator $F : \mathcal{C} \rightarrow \mathcal{D}$
2-cell	$R \sqsubseteq S$	a LaT $\varphi : \mathbf{G} \Rightarrow \mathbf{F} \triangleq \text{lax}$
order	$R \sqsubseteq S$	$\varphi \sqsubseteq \psi$ componentwise
union	$R \cup S$	$\varphi \cup \psi$, survives (13.2c)
product	$R \times S \triangleq \times$	$\varphi \times \psi$, survives; a TENSOR, not a categorical product; SYMMETRIC fork-proj (13.2c)
coproduct	$R + S$	$\varphi + \psi$, survives; a BIPRODUCT (13.2c)
meet	$R \cap S$	$\varphi \cap \psi$ componentwise, fails (13.2.1a)
converse	R°	φ° , fails, oplax: $\varphi_A^\circ \circ G(R) \sqsubseteq F(R) \circ \varphi_B^\circ$
zero object	z with $1 \ z = 0$	the constant relator at such a z , initial and terminal

§13.2.3. RES is a special case of LaT

(13.2.3a)



$G \triangleq \text{const } A$, $F \triangleq \text{const } B$ — the relators $X \rightarrow A$ and $X \rightarrow B$, each sending every arrow to $1 \triangleq \text{lax}$
 with $\mathcal{C}$ two objects and one non-identity arrow $X \rightarrow Y$, a LaT $\text{const } A \Rightarrow \text{const } B$ is the pair $\varphi_X, \varphi_Y : A \rightarrow B$ with $\varphi_Y \sqsubseteq \varphi_X$ — the datum RES

not every LaT is constant: $\exists : \mathbf{P} \Rightarrow \text{Id}$ powrel-laws $\pi_2 : (A \times -) \Rightarrow \text{Id}$ subseq-outr-square $\triangleleft : \text{Id} \Rightarrow \Delta$ rel-monoid

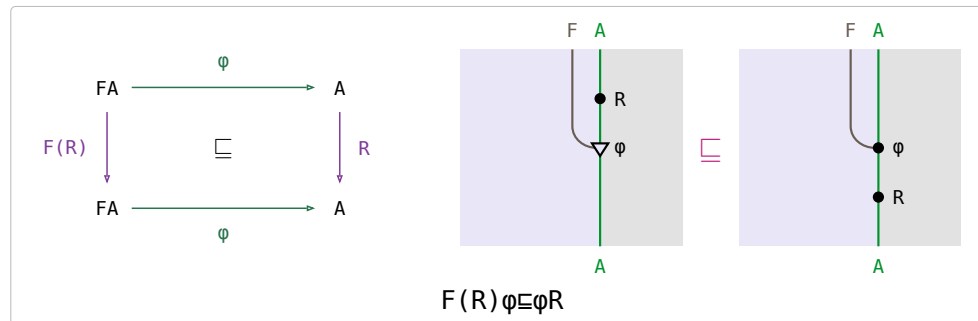
§13.3. Monotonic algebras

(13.3a)

An F -algebra $\varphi : FA \rightarrow A$ is **monotonic on** $R : A \rightarrow A$ when it is lax at R at $G := F$, $F := \text{Id}$: $F(R) \varphi \sqsubseteq \varphi R$. R is an **endorelation**, so $B = A$ and the two components φ_A, φ_B are the one arrow φ — an algebra, not a family.

For a map $f : FA \rightarrow A$ that is $f \circ F(R) f \in R$ adj-all's $f \circ \cdot \dashv f \cdot$ at $X := F(R) f, Y := R$, equivalently $F(R) \sqsubseteq f R f \circ \cdot \dashv f \cdot$ then $f \circ \cdot \dashv f \cdot, .$
 $(\leq \times \leq) + \sqsubseteq + \leq$ — addition on Nat is monotonic on $\leq$, which at the point level reads $c = a + b \wedge a \leq a' \wedge b \leq b' \implies c \leq a' + b'$.

(13.3b)



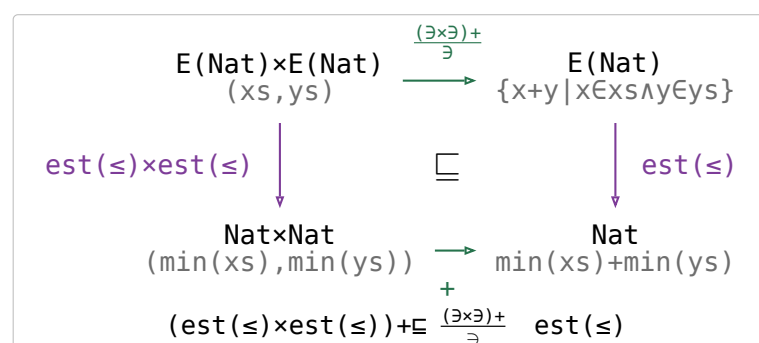
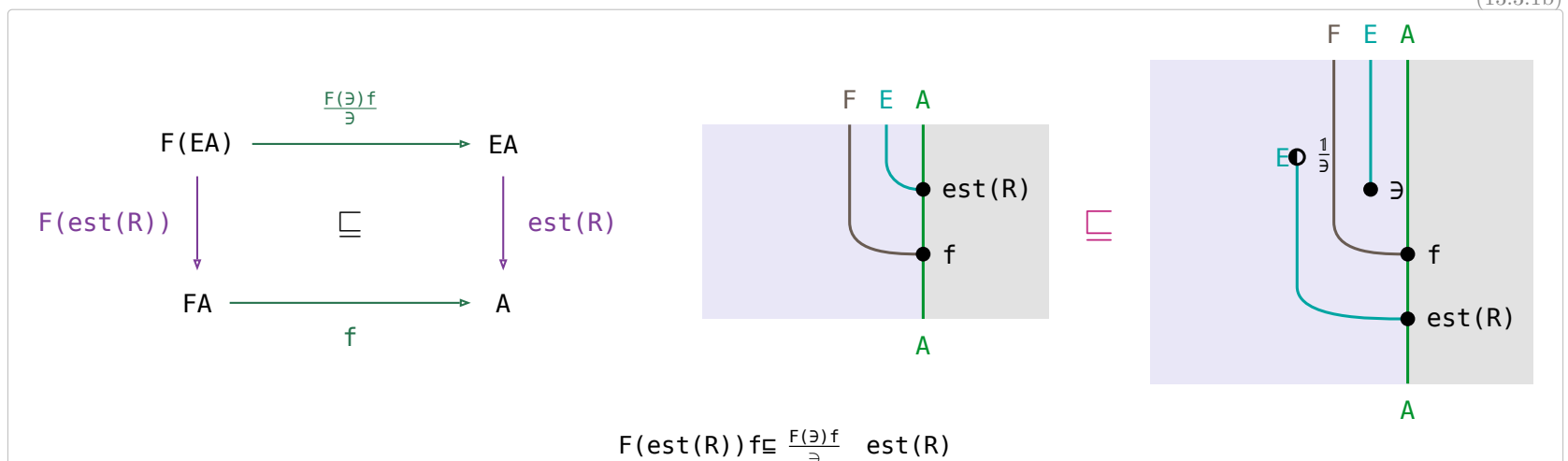
§13.3.1. Function f is monotonic on R iff it distributes over R

(13.3.1a)

$f : FA \rightarrow A$ **distributes over** R if $F(\text{est}(R)) f \sqsubseteq \frac{F(\exists)f}{\exists} \text{est}(R)$.

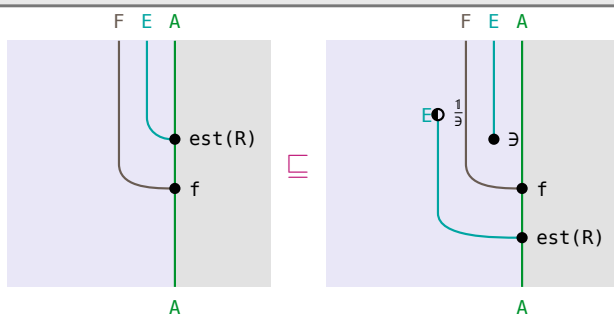
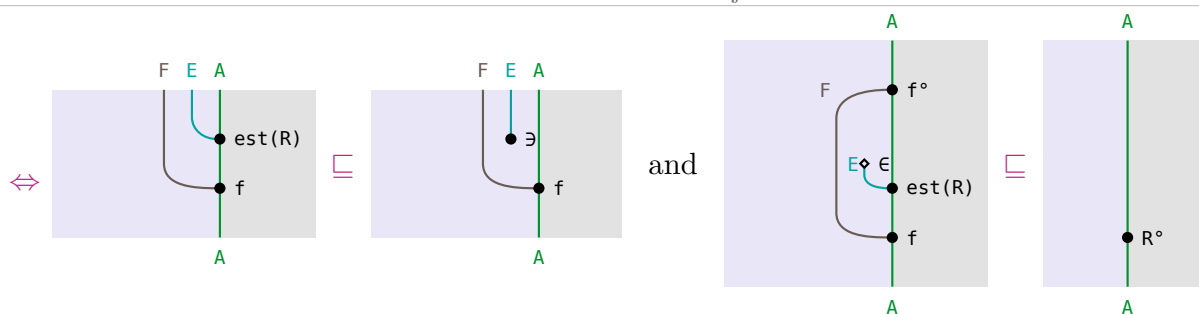
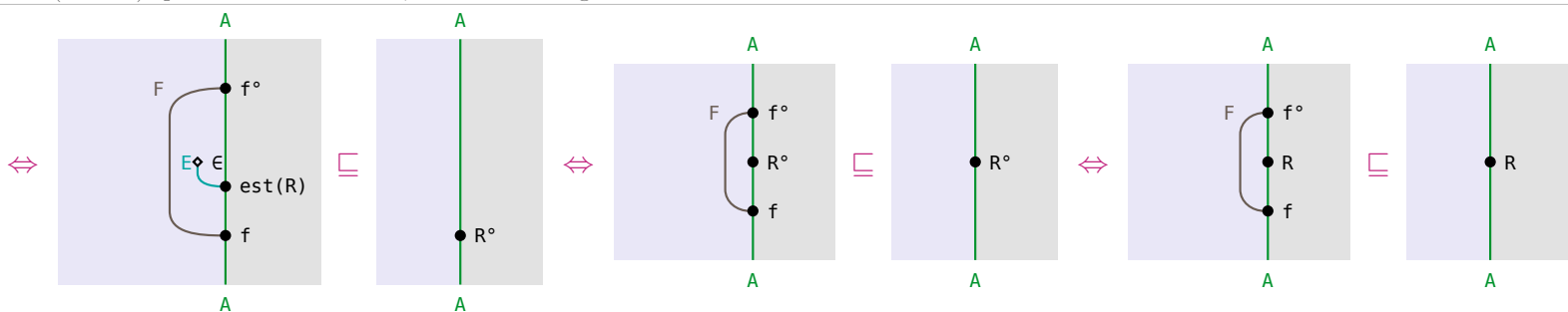
$+$ distributes over $\leq$, at the point level $\min(xs) + \min(ys)$ is a least element of $\{x + y \mid x \in xs \wedge y \in ys\}$ for xs, ys non-empty and $\min \triangleq \text{est}(\leq)$.

(13.3.1b)



$$f^\circ F(R) f \sqsubseteq R \iff F(\text{est}(R)) f \sqsubseteq \frac{F(\exists) f}{\exists} \text{est}(R)$$

(13.3.1c)

 function f is monotonic over R if and only if it distributes over R ; f a map, R reflexive

 f distributes over $R \iff$ distributes — the fraction bent as adj-E-bend

 (13.1.5a) splits the bound in two, div-laws moving $(F(\exists)f)^\circ$ across

 $\text{est}(R) \sqsubseteq \exists \iff \text{est}$ — so the first conjunct drops

 $(F(\exists)f)^\circ = f^\circ F(\epsilon) \iff \circ$ — and ϵ
 $\text{est}(R) = R^\circ \iff \text{est}$, R reflexive

 both sides conversed — $F(R^\circ)^\circ = F(R)$,
 relator-laws

circuit	Hinze–Marsden
$\langle \frac{S}{\triangleright} \text{est}(R) \rangle \sqsubseteq \langle \frac{\langle S \rangle}{\triangleright} \text{est}(R) \rangle$ the conclusion: one minimum kept at each step is below every result collected and one minimum taken at the end	
$\langle \frac{S}{\triangleright} \text{est}(R) \rangle \sqsubseteq \langle S \rangle$ and $\langle S \rangle^\circ \langle \frac{S}{\triangleright} \text{est}(R) \rangle \sqsubseteq R^\circ$ (13.1.5a) at $\langle S \rangle$; $\langle \frac{S}{\triangleright} \text{est}(R) \rangle \sqsubseteq \langle S \rangle^\circ \setminus R^\circ \iff \langle S \rangle^\circ \langle \frac{S}{\triangleright} \text{est}(R) \rangle \sqsubseteq R^\circ$ — div-laws	
$\frac{S}{\triangleright} \text{est}(R) \sqsubseteq S$ the left conjunct; (13.1.5a), and $\langle - \rangle$ is monotone	
 the right conjunct; $\langle S \rangle^\circ \langle \frac{S}{\triangleright} \text{est}(R) \rangle$ is the least X with $X = S^\circ F(X) (\frac{S}{\triangleright} \text{est}(R))$ hylomorphism theorem — so Knaster–Tarski leaves this one inequation $\frac{S}{\triangleright} = \frac{1}{\triangleright} E(S)$ — adj-E-bend	
 $S^\circ F(R^\circ) \sqsubseteq R^\circ S^\circ$ — $\triangleq$ monotonic at S , conversed; $F(R)^\circ = F(R^\circ)$ — relator-laws	
 $S^\circ (\frac{S}{\triangleright} \text{est}(R)) \sqsubseteq S^\circ (S^\circ \setminus R^\circ) \sqsubseteq R^\circ$ — (13.1.5a), div-laws	
 R transitive	

§13.3.3. $\text{takewhile}(p) = ([\text{nil}, (\lambda x. p \rightarrow \text{cons}, \rightarrow \text{nil})])$

(13.3.3a)

name	definition	type	example	in words
F	$FX = 1 + A \times X$	$\mathcal{A} \rightarrow \mathcal{A}$		nothing, or a head and a tail
nil, cons	$[A] ::= \text{nil} \mid \text{cons}(A, [A])$ $\triangleq \text{subseq}$	$1 \rightarrow [A],$ $A \times [A] \rightarrow [A]$	$\text{cons}(3, [1, 2]) = [3, 1, 2]$	the empty list; a head onto a tail
α	$[\text{nil}, \text{cons}]$	$F[A] \rightarrow [A]$		both constructors as one map
p	a coreflexive	$A \rightarrow A$	$p \triangleq \text{even} \rightarrow 2 \rightarrow p \rightarrow 2$, and $3 \notin \text{dom}(p)$	$\{(a, a) \mid a \text{ passes the test}\}$
R	$\text{length} \leq \text{length}^\circ$, a preorder	$[A] \rightarrow [A]$	$[1] R [1, 2]$	$xs R ys \iff \text{length}(xs) \leq \text{length}(ys)$
$\rightarrow \text{nil}$	the constant nil — the second nil of prefix	$A \times [A] \rightarrow [A]$	$(\rightarrow \text{nil})(3, [1, 2]) = \text{nil}$	drop the pair, return nil
prefix	$([\text{nil}, \rightarrow \text{nil} \cup \text{cons}]) \triangleq \text{subseq}$	$[A] \rightarrow [A]$	$[3, 1, 2] \text{ prefix } [3, 1]$	$xs \text{ prefix } ys \iff \exists zs. xs = ys \# zs$ — at each cons, stop or keep the head
S	$[\text{nil}, \rightarrow \text{nil} \cup (p \times 1) \text{ cons}]$	$F[A] \rightarrow [A]$	$(4, [2]) S [4, 2]$, and $(3, [2]) S \text{ nil only}$	prefix's algebra with one extra p — stop, or keep a head that passes p
$(g \rightarrow X, Y)$	X where g is defined and Y where it is not	$A \rightarrow B$		the test picks the branch

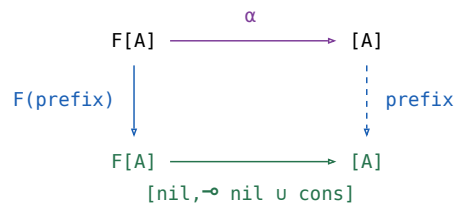
$\text{nil} R = \tau$, $\text{nil} R^\circ = \text{nil}$ nil is the shortest list — below every list, and above only itself, so it loses every $\text{est}(R^\circ)$

prefix = $([\text{nil}, \rightarrow \text{nil} \cup \text{cons}])$

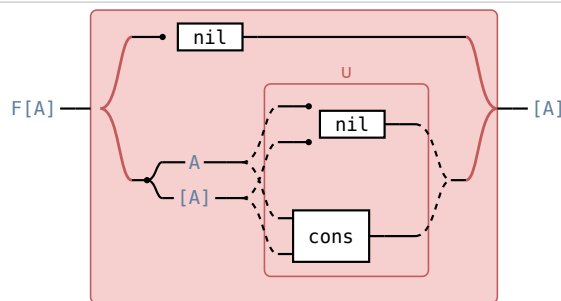
(13.3.3b)

the fold whose algebra, at each **cons**, stops with **nil** or keeps the head: $xs \text{ prefix } ys \iff \exists zs. xs = ys \# zs$

the defining square $\alpha \text{ prefix} = F(\text{prefix})[\text{nil}, \rightarrow \text{nil} \cup \text{cons}]$ — build the list and then take a prefix, or take a prefix of the tail and then rebuild with the algebra



$F = 1 + A \times -$, so $F(\text{prefix}) = 1 + 1 \times \text{prefix}$: the head passes, the fold recurses on the tail alone



the algebra $[\text{nil}, \rightarrow \text{nil} \cup \text{cons}] : F([A]) \rightarrow [A]$ as a circuit: **nil** on the **1** branch; on a pair (a, ys') two outputs, **nil** and $\text{cons}(a, ys')$

Hinze–Marsden



the **cons** operand of $\rightarrow \text{nil} \cup \text{cons}$; $\rightarrow \text{nil}$ makes a constant and draws nothing

$\text{nil prefix } ys \iff ys = \text{nil}$
 $\text{cons}(a, xs') \text{ prefix } ys \iff ys = \text{nil} \vee \exists ys'. xs' \text{ prefix } ys' \wedge ys = \text{cons}(a, ys')$

a right fold: α° peels, $F(\text{prefix})$ recurses, the algebra runs on the way back; $\text{init}^* (\triangleq \text{subseq})$ is the tail-recursive form
 lax natural only, $\text{list}(R) \text{ prefix} \sqsubseteq \text{prefix}$
 $\text{list}(R): \text{list}(R)$ first needs an R -image of every element, **prefix** first may have dropped the ones without

$\alpha \text{prefix list}(p) = F(\text{prefix list}(p))S$ (13.3.3c) building the list and then keeping a p -passing prefix of it is keeping one of the tail first, and then building with S this same diagram is subseq 's: algebra $[\text{nil}, \pi_2 \cup \text{cons}]$, type $[A] \rightarrow [A]$	
circuit — the fork is $F([A]) = 1 + A \times [A]$: nil above, the pair below	Hinze–Marsden
$\alpha \text{ prefix list}(p)$	the cons branch alone, without $1 +$ or $\multimap \text{nil}$
$F(\text{prefix}) [\text{nil}, \multimap \text{nil} \cup \text{cons}] \text{list}(p)$ defining equation	the cons operand of $\multimap \text{nil} \cup$
$F(\text{prefix}) [\text{nil}, \multimap \text{nil} \cup (p \times \text{list}(p)) \text{cons}]$ $\text{list}(p)$ through cons	the $(p \times \text{list}(p))$ cons operand of $\multimap \text{nil} \cup$ $(p \times \text{list}(p)) \text{cons}$
$[\text{nil}, \multimap \text{nil} \cup (p \times (\text{prefix list}(p))) \text{cons}]$ relator, prefix entire	
$F(\text{prefix list}(p))S$ $\text{prefix list}(p)$ entire	

$\triangleq$ **reduce** reads that off as $\text{prefix list}(p) = \{S\}$. fusion cannot: $\text{list}(p)$ is not entire, $(1 \times \text{list}(p)) \multimap \text{nil} \sqsubseteq \multimap \text{nil}$, and no algebra meets the side condition. ,

$(1 \times R^\circ)(\multimap \text{nil} \cup (p \times 1)\text{cons}) \sqsubseteq (\multimap \text{nil} \cup (p \times 1)\text{cons})R^\circ$ (13.3.3d)	
shortening the tail and then taking the step lands inside taking the step and then shortening the result	
circuit — the cons branch of $F(R^\circ)S \sqsubseteq SR^\circ$	reason
$(1 \times R^\circ)(\multimap \text{nil} \cup (p \times 1)\text{cons})$	
	each operand is reached on its own <i>adj-all</i> — and $(1 \times R^\circ)$ $(p \times 1)$ is p and R° on the pair's two strands at once
$(1 \times R^\circ)\multimap \text{nil} \cup (p \times R^\circ)\text{cons}$	
	$\multimap$ is the greatest arrow into 1 , so $(1 \times R^\circ)\multimap \sqsubseteq \multimap$
	$\text{cons length} = (1 \times \text{length})\pi_2$ succ with succ monotone — a shorter tail makes a shorter list
$\multimap \text{nil} \cup (p \times 1)\text{cons} R^\circ$	
	$\text{nil } R^\circ = \text{nil} \triangleq \text{takewhile}$ — so the constant branch may carry the R° the other one already has
$\multimap \text{nil } R^\circ \cup (p \times 1)\text{cons} R^\circ$	
	one R° past the join is the two inside it <i>adj-all</i>
$(\multimap \text{nil} \cup (p \times 1)\text{cons})R^\circ$	

the nil branch, which no row above draws, is $\text{nil} \sqsubseteq \text{nil } R^\circ$.

$\frac{S}{S} \text{ est}(R^\circ) = [\text{nil}, (\pi_1 p \rightarrow \text{cons}, \neg \text{nil})]$ (13.3.3e)	
the longest of the lists the algebra allows is the cons where the head passes p , and nil where it does not	
formula	reason
$ \begin{array}{c} A \\ \downarrow \\ [A] \xrightarrow{\frac{1}{S}} \text{EF}[A] \xrightarrow{E(S)} E[A] \xrightarrow{\text{est}(R^\circ)} [A] \end{array} $	
$ \begin{array}{c} = \\ F[A] \xrightarrow{\quad} \left(\begin{array}{c} \xrightarrow{\frac{1}{S}} E1 \xrightarrow{E(\text{nil})} E[A] \xrightarrow{\text{est}(R^\circ)} [A] \\ \xrightarrow{\frac{A}{[A]}} \xrightarrow{\frac{1}{S}} E(A \times [A]) \xrightarrow{E(\neg \text{nil} \cup (p \times 1) \text{cons})} E[A] \xrightarrow{\text{est}(R^\circ)} [A] \end{array} \right) \end{array} $ $ \left[\frac{\text{nil}}{S} \text{ est}(R^\circ), \frac{\neg \text{nil} \cup (p \times 1) \text{cons}}{S} \text{ est}(R^\circ) \right] $	coproduct of maps
$ \begin{array}{c} = \\ F[A] \xrightarrow{\quad} \left(\begin{array}{c} \xrightarrow{\text{nil}} [A] \\ \xrightarrow{\frac{A}{[A]}} \xrightarrow{\frac{1}{S}} E(A \times [A]) \xrightarrow{E(\neg \text{nil} \cup (p \times 1) \text{cons})} E[A] \xrightarrow{\text{est}(R^\circ)} [A] \end{array} \right) \end{array} $ $ [\text{nil}, \frac{\neg \text{nil} \cup (p \times 1) \text{cons}}{S} \text{ est}(R^\circ)] $	singleton, R° reflexive
$ \begin{array}{c} = \\ F[A] \xrightarrow{\quad} \left(\begin{array}{c} \xrightarrow{\text{nil}} [A] \\ \xrightarrow{\frac{A}{[A]}} \xrightarrow{\frac{1}{S}} E(A \times [A]) \xrightarrow{E(\neg \text{nil} \cup (p \times 1) \text{cons})} E[A] \xrightarrow{\text{est}(R^\circ)} [A] \end{array} \right) \end{array} $ $ [\text{nil}, \frac{\neg \text{nil} \cup (p \times 1) \text{cons}}{S} \text{ est}(R^\circ)] $	$\text{nil } R = \text{T}$

the set is $\{\text{nil}\}$ where p fails on the head and $\{\text{nil}, \text{cons}(a, xs)\}$ where it holds, and nil loses the second — $\triangleq \text{est}$ at a two-element set.

(13.3.3f)

name	definition	type	example	in words
S	$[\text{nil}, \neg \text{nil} \cup (p \times 1) \text{cons}] \triangleq \text{takewhile}$	$F[A] \rightarrow [A]$	$(4, [2]) \text{ S } [4, 2]$, and $(3, [2]) \text{ S nil only}$	prefix's algebra with one extra p — stop, or keep a head that passes p

$\text{takewhile}(p) \triangleq \frac{\text{prefix list}(p)}{S} \text{ est}(R^\circ)$ (13.3.3g)	
$\text{takewhile}(p) = ([\text{nil}, (\pi_1 p \rightarrow \text{cons}, \neg \text{nil})])$	
takewhile: $\text{takewhile}(p)(x)$ returns the longest prefix of x with the property that all its elements satisfy p ; the catamorphism is the standard implementation.	
$ \begin{array}{c} \text{list } A \\ \downarrow \\ \text{EO } \frac{1}{S} \xrightarrow{\quad} \nabla \text{prefix} \xrightarrow{\quad} \bullet p \xrightarrow{\quad} \bullet \text{est}(R^\circ) \end{array} $ the specification — $\triangleq \text{est}$'s (13.3.3c)	$ \begin{array}{c} \text{list } A \\ \downarrow \\ \text{EO } \frac{1}{S} \xrightarrow{\quad} \bullet (S) \xrightarrow{\quad} \bullet \text{est}(R^\circ) \end{array} $ greedy theorem at R°, with $F(R^\circ) \text{ S } \text{SR}^\circ$ — (13.3.3d) — for its hypothesis: one longest p-prefix kept at each cons, instead of every p-prefix collected and one chosen at the end.

$\text{takewhile}(p)^\circ \text{ takewhile}(p) \subseteq \text{prefix}^\circ \text{ prefix} \cap R \cap R^\circ \subseteq 1$
 $\text{takewhile}(p) \subseteq \text{prefix list}(p)$ and $(\text{prefix list}(p))^\circ \text{ takewhile}(p) \subseteq R$ — (13.1.5a) at $\text{est}(R^\circ)$ — and two prefixes of one list of equal length are equal, so $\text{takewhile}(p)$ is simple: the longest, not a longest.

$\frac{\text{prefix list}(p)}{S} \text{ est}(R^\circ)$ entire
 nil is always a p -prefix and R is connected on the prefixes of one list, so the longest exists.

$X \subseteq Y$, X entire, Y simple $\implies X = Y$

$[\text{nil}, (\pi_1 p \rightarrow \text{cons}, \neg \text{nil})]$ is a reduce of maps, hence entire — what turns the $\supseteq$ above into the heading's $=$.

§13.3.4. $mss = ([zero(1, \frac{1}{\exists}), ((1 \times \pi_1) \oplus, ((1 \times \pi_1) \oplus \frac{1}{\exists}, \pi_2 \pi_2) \text{ cup})]) \circ \pi_2 \text{ est}(\geq)$

(13.3.4a)

$FX = 1 + A \times X$, $\alpha \triangleq [nil, cons]$, $sum = ([zero, plus])$ and $segment = suffix \text{ prefix}$ from cata-examples and $\triangleq$ subseq.

$head \triangleq cons \circ \pi_1$, $wrap \triangleq (1, \rightarrow nil) \text{ cons}$ the head of a list and the one-element list, beside $\triangleq$ subseq's $tail \triangleq cons \circ \pi_2$

$\oplus \triangleq \frac{\rightarrow zero \cup plus}{\exists}$ $est(\geq)$ the set at (a, b) is $\{0, a+b\}$, so $\oplus$ is the larger of the two,

$mss = \frac{suffix}{\exists} E(\frac{prefix \text{ sum}}{\exists} \text{ est}(\geq)) \text{ est}(\geq)$ (13.3.4b)	
maximum segment sum problem: using segment=suffix prefix , the specification is expressed in this form	
formula — one wire from [A] to A, its type written along it	reason
$[A] \xrightarrow{\frac{1}{\exists}} E[A] \xrightarrow{E(\text{segment sum})} EA \xrightarrow{\text{est}(\geq)} A$	
$= [A] \xrightarrow{\frac{1}{\exists}} E[A] \xrightarrow{E(\text{suffix prefix sum})} EA \xrightarrow{\text{est}(\geq)} A$	segment=suffix prefix $\triangleq$ subseq, $\triangleq$ mss
$= [A] \xrightarrow{\frac{1}{\exists}} E(\text{suffix}) \xrightarrow{E[A]} E(\text{prefix sum}) \xrightarrow{EA} \text{est}(\geq) \xrightarrow{A}$	absorption pow-laws — $\frac{S}{\exists} = R$ $E(R) = \frac{SR}{\exists}$ at $S := suffix, R := prefix$ sum
$= [A] \xrightarrow{\frac{1}{\exists}} E(\text{suffix}) \xrightarrow{E[A]} E(\frac{prefix \text{ sum}}{\exists}) \xrightarrow{EEA} \text{union} \xrightarrow{EA} \text{est}(\geq) \xrightarrow{A}$	$\frac{R}{\exists} = R$, $\text{union} = E(\exists)$ pow-laws's $\frac{R}{\exists} = R$ at $R := prefix \text{ sum}$ and $E(R) \triangleq \frac{\exists R}{\exists}$; (13.1b)'s $\text{union} \triangleq \frac{\exists \exists}{\exists}$; the middle equality is $\triangleq$ relator's $F(RS) = F(R)F(S)$ at $F := E$
$= [A] \xrightarrow{\frac{1}{\exists}} E(\text{suffix}) \xrightarrow{E[A]} E(\frac{prefix \text{ sum}}{\exists}) \xrightarrow{EEA} E(\text{est}(\geq)) \xrightarrow{EA} \text{est}(\geq) \xrightarrow{A}$	(13.1b), the sets non- empty
$= [A] \xrightarrow{\frac{1}{\exists}} E(\text{suffix}) \xrightarrow{E[A]} E(\frac{prefix \text{ sum}}{\exists} \text{ est}(\geq)) \xrightarrow{EA} \text{est}(\geq) \xrightarrow{A}$	relator $\triangleq$ relator — $F(RS) = F(R)F(S)$ at $F := E$

The **union** step is (13.1b)'s $P(\text{est}(R)) \text{ est}(R) = P(\text{dom}(\text{est}(R))) \text{ union } \text{est}(R)$ — every suffix has the empty prefix, so **Dom** is 1 here — and $P(f) = E(f)$ at the map it is applied to (powrel-laws).

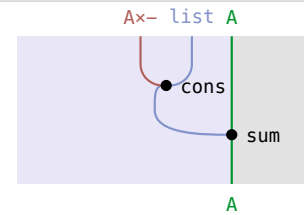
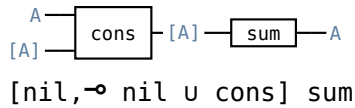
$$[\text{nil}, \multimap \text{nil} \text{ u } \text{cons}] \text{sum} = F(\text{sum})S$$

(13.3.4c)

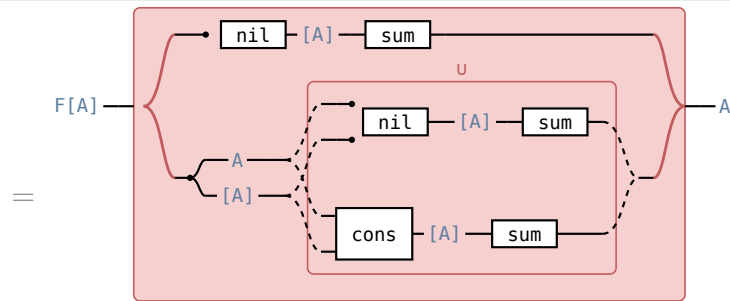
fusion: **prefix** expressed as a catamorphism on cons-lists, $([\text{nil}, \multimap \text{nil} \text{ u } \text{cons}])$, and this is the fusion condition that expresses **prefix sum** as a catamorphism

circuit — the fork is the bracket's case split $F([A]) = 1 + A \times [A]$: nil above, the pair and its u below

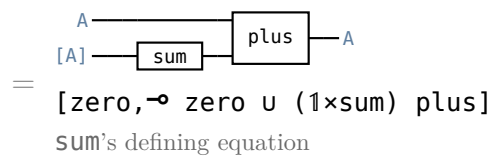
Hinze–Marsden



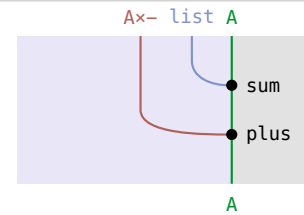
the **cons** operand of $\multimap \text{nil} \text{ u } \text{cons}$



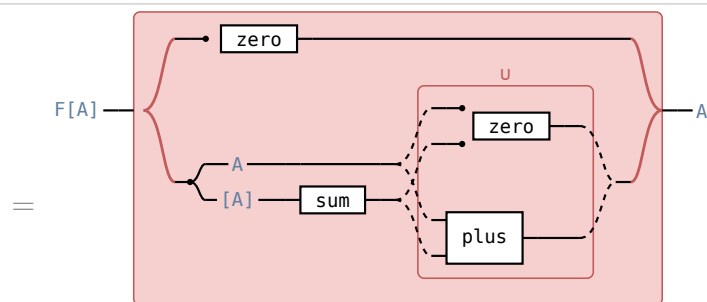
coproduct of maps, composition over U



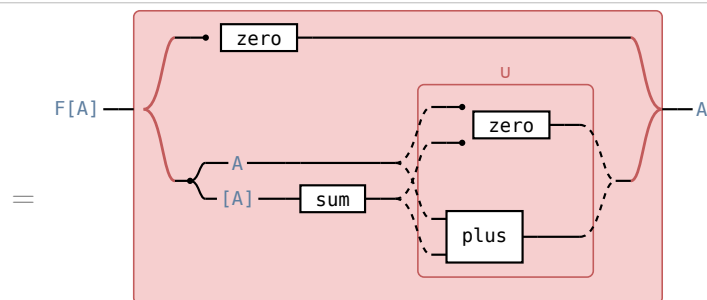
sum's defining equation



the $(1 \times \text{sum})$ **plus** operand of $\multimap \text{zero} \text{ u } (1 \times \text{sum})$ **plus**



$(1 \times \text{sum}) \multimap = \multimap$, sum entire



relator

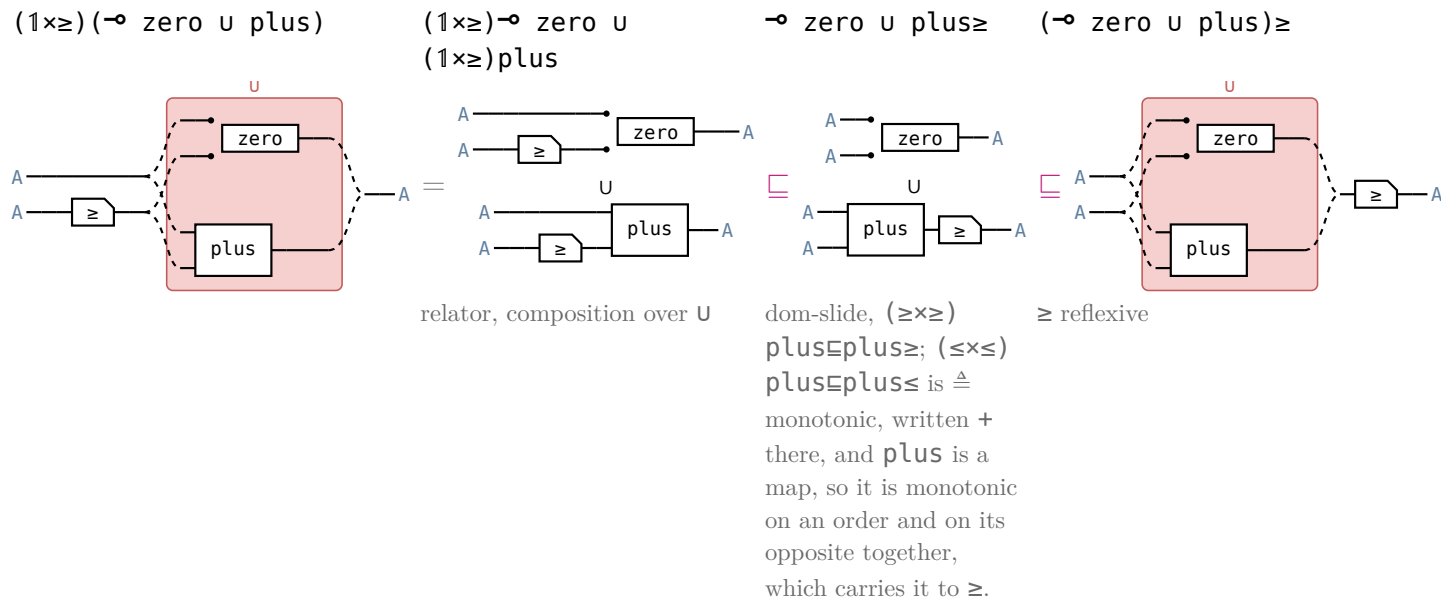
fusion at $R := [\text{nil}, \multimap \text{nil} \text{ u } \text{cons}]$, $S := \text{sum}$: the side condition, so **prefix sum** $= ([\text{zero}, \multimap \text{zero} \text{ u } \text{plus}])$. **prefix** is the reduce, **sum** the map fused into it — the intermediate list is gone.

$$(1 \times \geq)(\neg \text{zero} \cup \text{plus})E(\neg \text{zero} \cup \text{plus})\geq$$

(13.3.4d)

monotonic algebra: an **F**-algebra **S** is monotonic on a relation **R** if $F(R)SESR$ — the **plus** branch of $F(\geq)SE\geq$; the **zero** branch is $\text{zero}E\text{zero}\geq$

circuit — the head above, the running sum below; the tape is the **u**



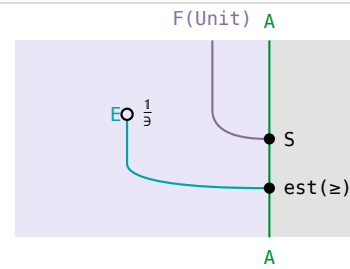
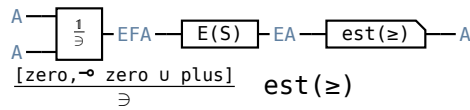
$$\frac{S}{\geq} \text{est}(\geq) = [\text{zero}, \oplus]$$

(13.3.4e)

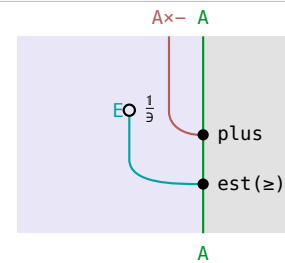
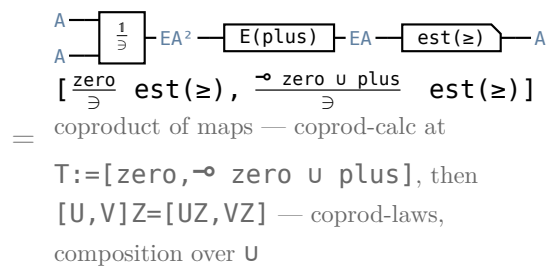
the largest sum the algebra offers is zero from nothing and, from a head and a running sum, the larger of zero and the head added to it

circuit — the tape is the coproduct:
zero's branch above, plus's below

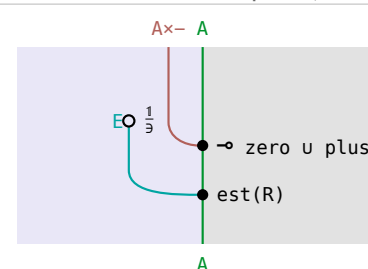
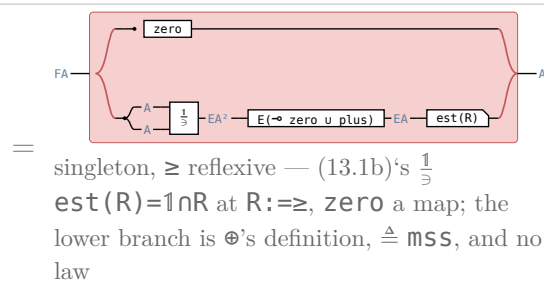
Hinze–Marsden



the **zero** arm of the bracket



the **plus** operand of the lower arm's $\neg \text{zero} \cup \text{plus}$, under its $1/3 E(\dots)$ and $\text{est}(\geq)$



with (13.3.4d) the greedy theorem gives $([\text{zero}, \oplus]) \sqsubseteq \frac{\text{prefix sum}}{\geq} \text{est}(\geq)$ — and (13.3.4g) makes it an equality.

$[\text{nil}, \text{cons}] (\langle \langle [\text{zero}, \oplus] \rangle, \frac{\text{suffix}}{\supset} E(\langle \langle [\text{zero}, \oplus] \rangle) \rangle) = F(\langle \langle [\text{zero}, \oplus] \rangle, \frac{\text{suffix}}{\supset} E(\langle \langle [\text{zero}, \oplus] \rangle) \rangle) k \quad (13.3.4f)$ $k \triangleq [\text{zero}(\frac{1}{\supset}), \langle w, \langle w \frac{1}{\supset}, \pi_2 \pi_2 \rangle \text{cup} \rangle], w \triangleq (1 \times \pi_1) \oplus: \text{the value at the whole list, paired with the set of the values at its suffixes, runs } k \text{'s recursion.}$	
the equation at that branch	why
$\text{nil}(g, \frac{\text{suffix}}{\supset} E(g)) = \text{zero}(\frac{1}{\supset})$	nil has one suffix, itself, and $g(\text{nil}) = \text{zero}$, so the set is the singleton $\{\text{zero}\}$
$\text{cons}(g, \frac{\text{suffix}}{\supset} E(g)) = (1 \times \langle g, \frac{\text{suffix}}{\supset} E(g) \rangle) \langle w, \langle w \frac{1}{\supset}, \pi_2 \pi_2 \rangle \text{cup} \rangle$	$g(\text{cons}(a, x)) = a \oplus (g(x))$, which is w reading $g(x)$ off π_1 ; and the suffixes of $\text{cons}(a, x)$ are $\text{cons}(a, x)$ itself, whose value is that same w , together with those of x , which $\pi_2 \pi_2$ carries — so the two sets meet at cup

$\text{mss} = (k) \pi_2 \text{est}(\geq) \quad (13.3.4g)$ maximum segment sum problem: $\frac{\text{suffix}}{\supset} E(\langle \langle [\text{zero}, \oplus] \rangle) \rangle$ expressed as the catamorphism $\langle k \rangle$, hence mss implemented by a linear-time algorithm, $\oplus \triangleq \frac{\text{zero} \cup \text{plus}}{\supset} \text{est}(\geq) \triangleq \text{mss}; k \text{ and } w \triangleq (13.3.4f).$	
mss is the greatest of the segment sums — $\triangleq \text{mss}$ (13.3.4b)	$\frac{\text{suffix}}{\supset} E(\langle \langle [\text{zero}, \oplus] \rangle) \rangle \text{est}(\geq) \triangleq \text{reduce at } (13.3.4f) \text{'s equation, so } \langle k \rangle = \langle \langle [\text{zero}, \oplus] \rangle, \frac{\text{suffix}}{\supset} E(\langle \langle [\text{zero}, \oplus] \rangle) \rangle \rangle$, of which π_2 is the row above the greedy theorem greedy theorem at $R := \geq, S := [\text{zero}, \text{plus}]$ — (13.3.4d) is its condition and (13.3.4e) its $\frac{S}{\supset} \text{est}(\geq)$; its $\sqsubseteq$ is an $=$ because $\langle [\text{zero}, \oplus] \rangle$ is entire and $\frac{\text{prefix sum}}{\supset} \text{est}(\geq)$ simple (13.3.3g)'s last row

one fold builds the $n+1$ running maxima and the final $\text{est}(\geq)$ reads them in one more pass, so mss is linear.

§13.3.5. $\text{filter}(p) = ([\text{nil}, (\pi_1 p \rightarrow \text{cons}, \pi_2)])$

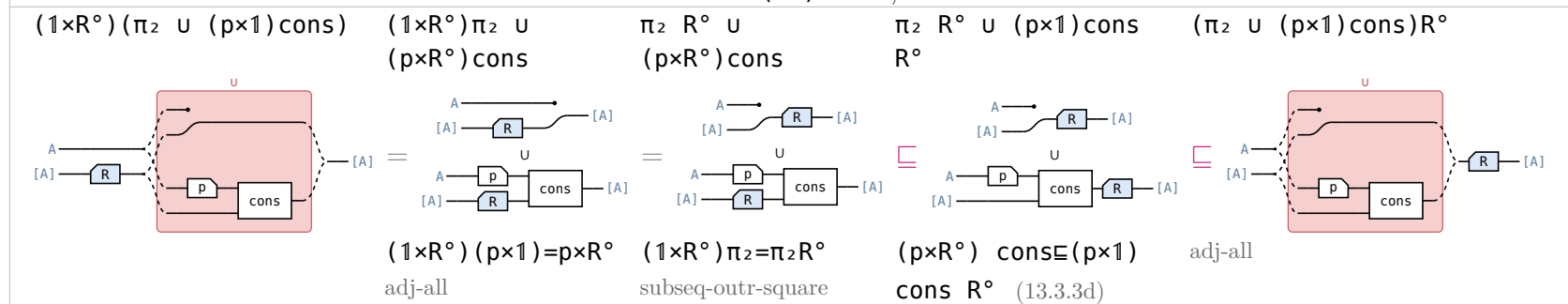
(13.3.5a)

name	definition	type	example	in words
F, α, p, R	as in $\triangleq \text{takewhile}$			
π_2	π_2 where $\triangleq \text{takewhile}$ has $\rightarrow \text{nil}$	$A \times [A] \rightarrow [A]$	$\pi_2(3, [1, 2]) = [1, 2]$	drop the head, keep the tail
subseq	$([\text{nil}, \pi_2 \cup \text{cons}]) \triangleq \text{subseq}$	$[A] \rightarrow [A]$	$[3, 1, 2] \text{ subseq } [3, 2]$	$xs \text{ subseq } ys \iff ys \text{ is } xs \text{ with elements dropped}$ — at each cons , drop the head or keep it
S	$[\text{nil}, \pi_2 \cup (p \times 1) \text{ cons}]$	$F[A] \rightarrow [A]$	$(4, [2]) S [2]$ and $(4, [2]) S [4, 2]$, but $(3, [2]) S [2]$ only	subseq's algebra with one extra p — drop the head, or keep a head that passes p
$1 \sqsubseteq \pi_2 R \text{ cons}^\circ$				the tail is one shorter than the cons , so π_2 loses the $\text{est}(R^\circ)$ at every step — where $\triangleq \text{takewhile}$'s loser is nil

$$(1 \times R^\circ)(\pi_2 \cup (p \times 1) \text{ cons}) \sqsubseteq (\pi_2 \cup (p \times 1) \text{ cons}) R^\circ$$

(13.3.5b)

shortening the tail and then taking the step lands inside taking the step and then shortening the result

formula — the cons branch of $F(R^\circ) S \sqsubseteq S R^\circ$; reason under each circuitthe nil branch: $\text{nil} \sqsubseteq \text{nil} R^\circ$.

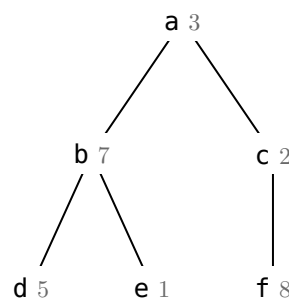
§13.4. Planning a company party

(13.4a)

definition	type	note
$\text{tree } A ::= \text{node } (A, [\text{tree } A])$	$\mathcal{A} \rightarrow \mathcal{A}$	The company hierarchy: an employee, and the list of subtrees under them.
$F(A, B) = A \times [B]$	$\mathcal{A} \rightarrow \mathcal{A}$	The base functor tree folds: an employee beside the recursive position, one layer deep.
rating	$A \rightarrow \text{Int}$	What one employee is worth as a guest.
$\text{cost} = \text{list}(\text{rating}) \text{sum}$	$[A] \rightarrow \text{Int}$	What a guest list is worth.
$R = \text{cost} \leq \text{cost}^\circ$	$[A] \rightarrow [A]$	The preorder the guest list is maximised over.
$\text{choose} \triangleq \pi_1 \cup \pi_2$	$[A] \times [A] \rightarrow [A]$	Takes one of the two parties a subtree returns.
$\text{include} = (1 \times (\text{list}(\pi_2) \text{concat})) \text{cons}$	$F(A, [A] \times [A]) \rightarrow [A]$	The party that invites the root, which puts every immediate subtree's root out. A map.
$\text{exclude} = (1 \times (\text{list}(\text{choose}) \text{concat})) \pi_2$	$F(A, [A] \times [A]) \rightarrow [A]$	The party that leaves the root out, so each subtree is free to choose. Not a map.
$S \triangleq (\text{include}, \text{exclude})$	$F(A, [A] \times [A]) \rightarrow [A] \times [A]$	The algebra: one step returns both parties of a subtree at once.
$\text{party} = (S) \text{choose}$	$\text{tree}(A) \rightarrow [A]$	Every guest list the president's ruling allows.
the specification $\frac{\text{party}}{\exists} \text{est}(R^\circ)$	$\text{tree}(A) \rightarrow [A]$	A guest list of greatest total conviviality.

§13.4.1. include and exclude

(13.4.1a)



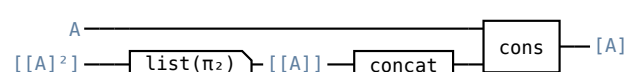
the small grey number is the employee's rating

(13.4.1b)

node	the algebra's input $A \times [[A] \times [A]]$	include	exclude
d, e, f	$(d, [])$	$[d]$	$[]$
b	$(b, ([[d], []], ([e], []]))$	$[b]$	$[d, e], [d], [e], []$
c	$(c, ([[f], []]))$	$[c]$	$[f], []$
a	$(a, ([[b], \dots], ([c], \dots)))$	$[a] \# \text{exclude}(b) \# \text{exclude}(c)$	$\text{choose}(b) \# \text{choose}(c)$

with those ratings, $\text{est}(R^\circ)$ keeps $([b]=7, [d, e]=6)$ at **b**, where **include** wins, and $([c]=2, [f]=8)$ at **c**, where **exclude** wins, so at the root $\text{include}=[a, d, e, f]=17$ beats $\text{exclude}=[b, f]=15$, and **choose** takes 17.

(13.4.1c)



(13.4.1d)

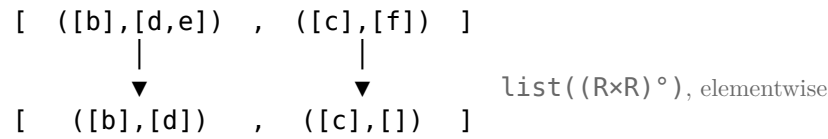


(13.4.1e)

$\text{list}(\text{choose}) \ [([d],[]), ([e],[])]$
 1st item $([d],[])$ choose $\rightarrow [d]$ or $[]$
 2nd item $([e],[])$ choose $\rightarrow [e]$ or $[]$
 2x2=4 combinations: $[[d],[e]] \quad [[d],[]] \quad [[],[e]] \quad [[],[]]$
 concat flattens: $[d,e] \quad [d] \quad [e] \quad []$

§13.4.2. $\text{list}((R \times R)^\circ)$

(13.4.2a)



$([b],[d,e])$
 $\begin{cases} \text{exclude: the best party in b's subtree when b does not come} \\ \text{include: the best party in b's subtree when b comes} \end{cases}$

(13.4.2b)

element	1st component include	2nd component exclude
1st, $([b], [d,e])$	$[b] R^\circ [b]$ $7 \geq 7$, reflexivity	$[d,e] R^\circ [d]$ $6 \geq 5$, and $([b], [d,e])$ is the pair $\text{est}(R^\circ)$ keeps at b
2nd, $([c], [f])$	$[c] R^\circ [c]$ $2 \geq 2$, reflexivity	$[f] R^\circ []$ $8 \geq 0$, and $([c], [f])$ is the pair $\text{est}(R^\circ)$ keeps at c

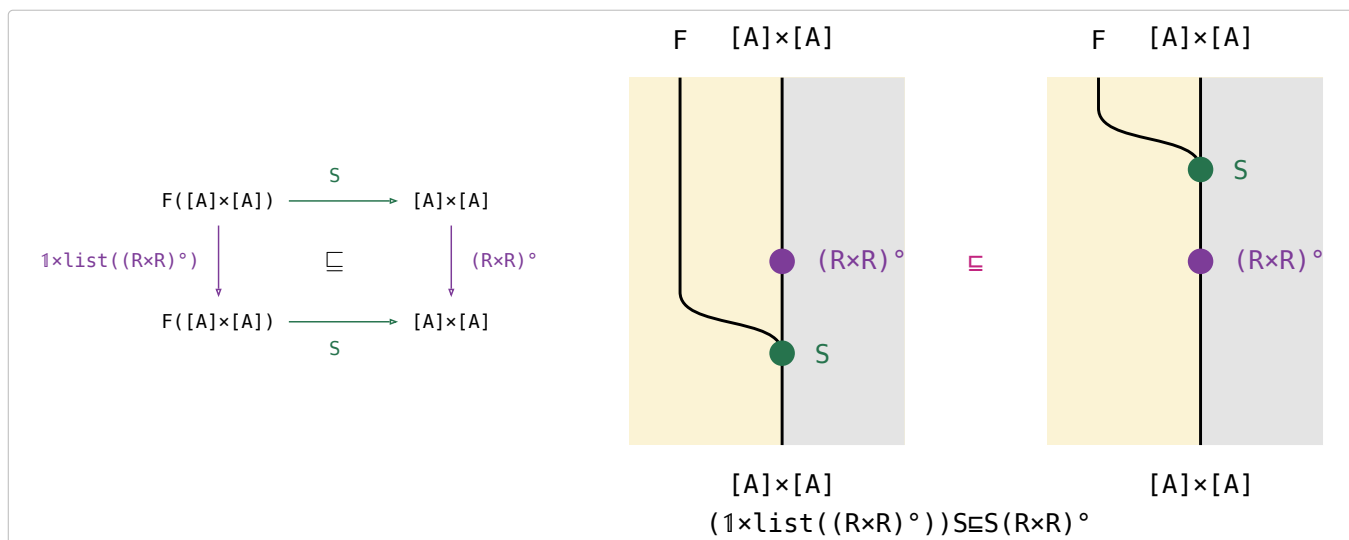
the first component of $1 \times \text{list}((R \times R)^\circ)$ is the root employee.

$R \triangleq \text{cost} \leq \text{cost}^\circ$ is a preorder, so the components that do not move are related by reflexivity — $(R \times R)^\circ$ demands a relation in **both** components, it cannot skip one.

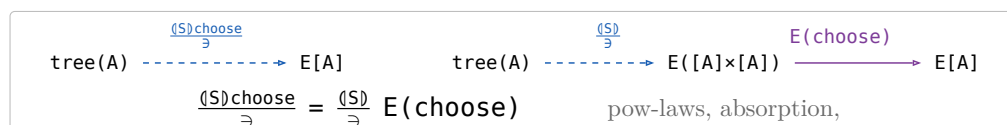
R compares cost only, not contents: $[f] R^\circ []$ is legal though $[f]$ has an element $[]$ does not. That is why the three leaves of §Section 13.4.3's proof all reduce to “**COST** is a sum”.

§13.4.3. $(1 \times \text{list}((R \times R)^\circ)) S \subseteq S(R \times R)^\circ \text{ — } S : F([A] \times [A]) \rightarrow [A] \times [A] \text{ monotonic on } (R \times R)^\circ$

(13.4.3a)

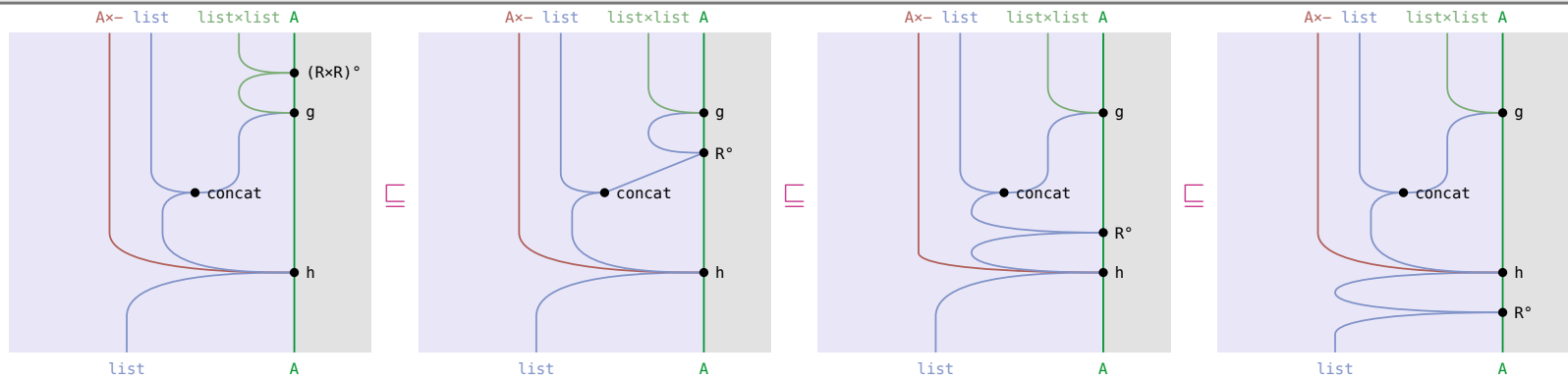


(13.4.3b)



$$\begin{aligned}
 S^{\Delta}(\text{include}, \text{exclude}) & \\
 (1 \times \text{list}((R \times R)^{\circ}))S & \sqsubseteq S(R \times R)^{\circ} \\
 (1 \times \text{list}((R \times R)^{\circ}))\text{include} & \sqsubseteq \text{include } R^{\circ} \\
 (1 \times \text{list}((R \times R)^{\circ}))\text{exclude} & \sqsubseteq \text{exclude } R^{\circ}
 \end{aligned}
 \tag{13.4.3c}$$

bettering both parties of every subtree before the node's algebra runs gets no further than running it first and bettering the two parties it returns,



- g $(R \times R)^{\circ} g \sqsubseteq g R^{\circ}$
 $g := \pi_2$ is $(R \times R)^{\circ} \pi_2 = (\text{dom}(\pi_1 R^{\circ})) \pi_2 R^{\circ} \sqsubseteq \pi_2 R^{\circ}$, $g := \pi_1$ its mirror $(\text{dom}(\pi_2 R^{\circ})) \pi_1 R^{\circ} \sqsubseteq \pi_1 R^{\circ}$ — 1 and 4 of bdm-prod-laws, then $\text{dom} \sqsubseteq 1$; $g := \text{choose}^{\Delta} \pi_1 \cup \pi_2$ is the union of the two (13.2c). **list** monotonic, Δ relator
- concat $\text{list}(R^{\circ}) \text{concat} \sqsubseteq \text{concat } R^{\circ}$
 a LEAF: no law above it. **cost** is a sum, so $\text{cost}(\text{concat}(xss)) = \text{sum}(\text{list}(\text{cost})(xss))$ and a cheaper part makes a cheaper whole.
- h $(1 \times R^{\circ}) h \sqsubseteq h R^{\circ}$
 a LEAF: $\text{cost}(\text{cons}(a, xs)) = \text{rating}(a) + \text{cost}(xs)$, so a cheaper tail makes a cheaper list. For $h := \pi_2$ it is an EQUALITY $(1 \times R^{\circ}) \pi_2 = (\text{dom}(\pi_1)) \pi_2 R^{\circ} = \pi_2 R^{\circ}$: π_1 is a map, hence entire, so $\text{dom}(\pi_1) = 1$ — dom-laws

$(1 \times (\text{list}(g) \text{ concat}))h$	g	h
include	π_2	cons
exclude	choose	π_2

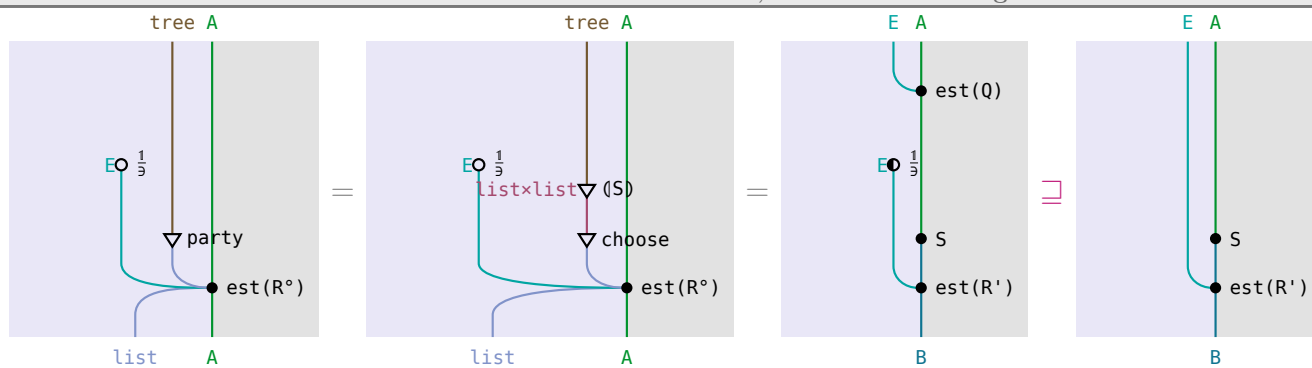
(13.4.3d)

$R^{\Delta} \{(\emptyset, \emptyset)\} : \{0, 1\} \rightarrow \{0, 1\}$	not entire — undefined at 1
$(R \times R) \text{ choose}$ at $(\emptyset, 1)$	nothing — $R \times R$ needs BOTH components related, and R is undefined at 1
$\text{choose } R$ at $(\emptyset, 1)$	0 — choose picks the first component 0, and $R \ 0 \ 0$ holds
$((\emptyset, 1), \emptyset)$	in choose R , not in $(R \times R) \text{ choose}$
$(R \times R) \text{ choose} = \text{choose } R$	iff R is entire — $\text{dom} \sqsubseteq 1$ is the g row's only inequality step, and it is an equality iff $\text{dom}(R) = 1$

§13.4.4. The derivation

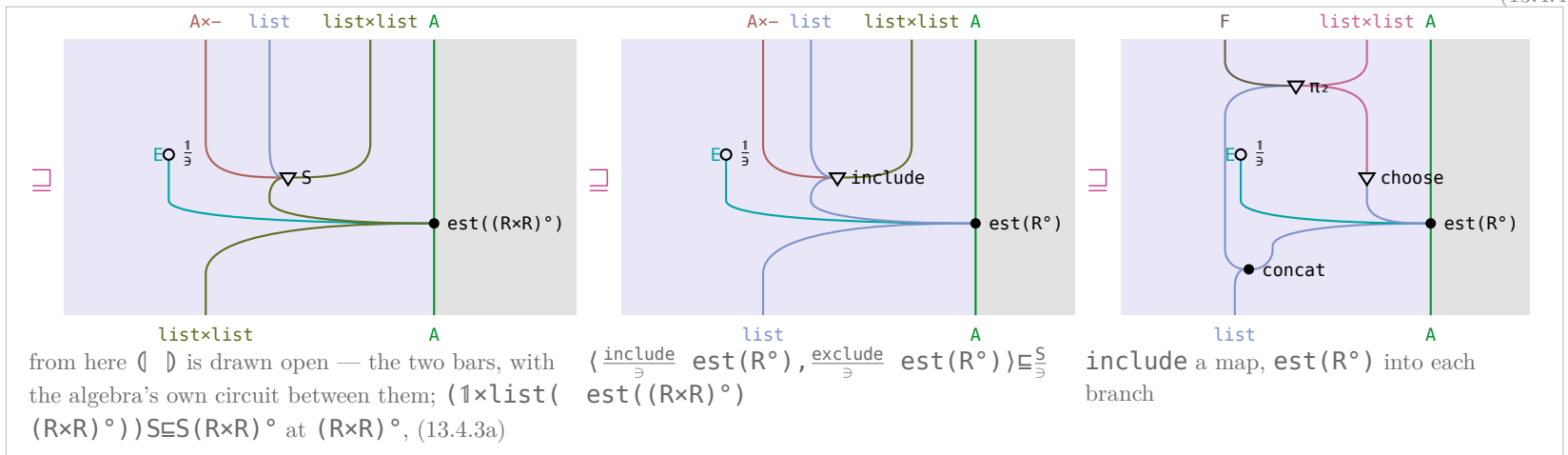
$$(\langle \text{include}, \pi_2 \text{ list}(\frac{\text{choose}}{\exists} \text{ est}(R^{\circ})) \text{ concat} \rangle) \frac{\text{choose}}{\exists} \text{ est}(R^{\circ}) \sqsubseteq \frac{\text{party}}{\exists} \text{ est}(R^{\circ})
 \tag{13.4.4a}$$

the best of every guest list the president allows is one pass up the tree, each subtree handing up its best party with its boss in and its best with the boss out, and **choose** taking the better of the two at the root



the specification — Δ part $\text{party}^{\Delta}(S)$ **choose** — Δ party $\frac{(S)}{\exists} \text{ choose} = \frac{(S)}{\exists} E(\text{choose})$ $(R \times R)^{\circ} \text{ choose} \sqsubseteq \text{choose } R^{\circ}$ — (13.4.3b) (13.4.3c)'s g row

(13.4.4b)



(13.4.4b)

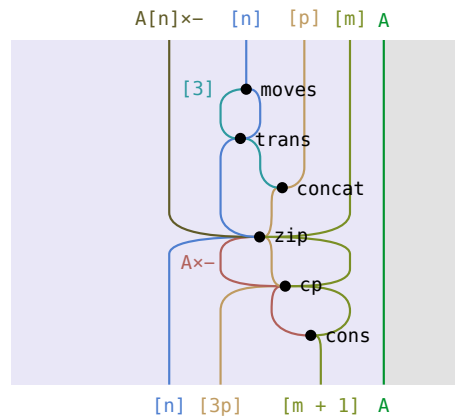
§13.5. Shortest paths on a cylinder

(13.5a)

	on $L = \text{list}^+$, sec-cyl-gen	on Vec	note
F	$F(A, X) = A + A \times X$	$F(A, A[m]) = A[1] + A[m+1]$	A square alone, or in front of the path so far.
the input	$L(N\ A)$	$A[m][n] \triangleq (A[n])[m]$	m columns of n squares, outermost index first — the matrix itself.
a path	$L\ A$, a non-empty list	$A[1] \triangleq A$, $A[m+1] \triangleq A \times A[m]$	Through m columns, m squares: the count is in the type. A one-square path is the square; $\text{Vec}(m)$ is the functor $X \mapsto X[m]$.
the paths into a row	$E(L\ A)$	$A[p][m]$, $p = 3^{m-1}$	Three moves per column crossed, so this count is in the type too; a set forgets it.
α	$[\text{wrap}, \text{cons}]$ $F(A, LA) \rightarrow LA$	$\text{wrap} = 1 : A \rightarrow A[1]$ $\text{cons} : A \times A[m] \rightarrow A[m+1]$	wrap is an identity; cons is only an iso, so it stays a bead.
moves	$NA \rightarrow E(NA)$	$X[n] \rightarrow X[3][n]$	$\text{moves}(x) = (\text{up}(x), x, \text{down}(x))$, a $3 \times n$ matrix.
trans	$E(NX) \rightarrow N(EX)$	$X[3][n] \rightarrow X[n][3]$	The transpose.
union	$E(EX) \rightarrow EX$	$\text{concat} : X[j][k] \rightarrow X[jk]$	The j rows laid end to end; j is 3 inside gen and n inside paths .
zip	$F(NA, NB) \rightarrow NF(A, B)$	$F(A[n], B[n]) \rightarrow F(A, B)[n]$	Unchanged.
cp	$F(A, EB) \rightarrow E(F(A, B))$	$A \times B[p] \rightarrow (A \times B)[p]$	The square paired with each of the p candidates.
$\text{est}(R) \triangleq \exists n (E \setminus R^p) X \rightarrow X$		$X[k] \rightarrow X$, $\exists : X[k] \rightarrow X$	A component that is R -below every component; $\exists$ is “is some component” and E its converse.
$R \triangleq \text{sum} \leq \text{sum}^\circ$	$L\ \text{Nat} \rightarrow L\ \text{Nat}$	$\text{Nat}[m] \rightarrow \text{Nat}[m]$	The cost of a path, which the cheapest minimises — one order per length m .
gen	$F(1, \text{moves trans } N(\text{union}))$ $\text{zip } N(\text{cp } P(\alpha))$ $F(NA, N(E(LA)))$ $\rightarrow N(E(LA))$	$F(1, \text{moves trans } \text{Vec}(n)(\text{concat}))$ $\text{zip } \text{Vec}(n)(\text{cp } \text{Vec}(3p)(\text{cons}))$ $F(A[n], A[n][p][m]) \rightarrow A[n][3p][m+1]$	The same composite without α ; the type shown is the cons side, the wrap side is $1 : A[n] \rightarrow A[n]$.
$\langle \text{gen} \rangle$	$L(N\ A) \rightarrow N(E(L\ A))$	$A[m][n] \rightarrow A[n][p][m]$	Columns in, rows out.
$\text{paths} \triangleq \langle \text{gen} \rangle$ concat	$\langle \text{gen} \rangle$ setify union $L\ N\ \text{Nat} \rightarrow E(L\ \text{Nat})$	$A[m][n] \rightarrow A[np][m]$	The n rows of p paths laid end to end: every path of the cylinder, in one row.
Q , Section 13.5.3	$F(1, \text{moves trans } N(\text{est}(R)))$ $\text{zip } N(\alpha)$	$F(1, \text{moves trans } \text{Vec}(n)(\text{est}(R)))$ $\text{zip}, F(A[n], A[n][m]) \rightarrow A[n][m+1]$	The cheapest of three; no p anywhere. $\langle Q \rangle : A[m][n] \rightarrow A[n][m]$ has the type of a transpose.
the specification paths $\text{est}(R)$	$L\ N\ \text{Nat} \rightarrow L\ \text{Nat}$	$A[m][n] \rightarrow A[m]$, $\text{est}(R) : X[np] \rightarrow X$	A cheapest path from the entry side to the exit side.

§13.5.1. $\text{gen} \triangleq (1 \times (\text{moves trans } \text{Vec}(n)(\text{concat}))) \text{ zip } \text{Vec}(n)(\text{cp } \text{Vec}(3p)(\text{cons}))$

(13.5.1a)



(13.5.1b)

a row is the outer index, a cell one path

$u = ((1,2,3,4), C)$

$C = 5$

6

7

8

$(1 \times (\text{moves trans Vec}(n)(\text{concat})))$

$\text{moves}(C)$

$= 6 \ 7 \ 8 \ 5$

5 6 7 8

8 5 6 7

$\text{trans}(\text{that})$

$= 6 \ 5 \ 8$

7 6 5

8 7 6

5 8 7

$\text{Vec}(n)(\text{concat})(\text{that}) = \text{that}$

$\text{zip}(\text{that})$

$= (1, 6 \ 5 \ 8)$

(2, 7 6 5)

(3, 8 7 6)

(4, 5 8 7)

$\text{Vec}(n)(\text{cp})(\text{that})$

$= 1 \ 6 \ 2 \ 7 \ 3 \ 8 \ 4 \ 5$

1 5 2 6 3 7 4 8

1 8 2 5 3 6 4 7

$: A[n] \times A[n]$

one candidate per row, one square long:

a column of squares

1 keeps the column, the candidates move down, unmoved, up: a $3 \times n$ matrix

$: A[3][n]$

the transpose: row k gets rows $k-1, k, k+1$

$: A[n][3]$

$\text{concat}: X[3][1] \rightarrow X[3]$ is 1

each row: its square, and its candidates

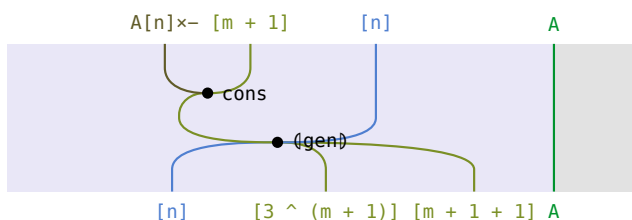
$: (A \times A[3])[n]$

cp pairs the square with each candidate:

row k of that is the k -th 3×2 block, left to right; each line a two-square path

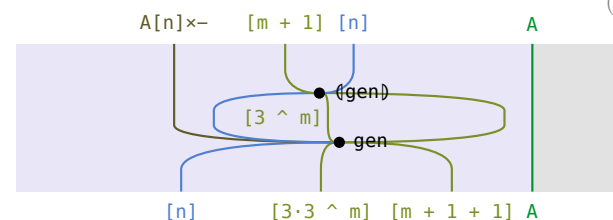
$: A[n][3][2]$

§13.5.2. gen is an F-algebra; $(\text{gen}): \text{cons } (\text{gen}) = (1 \times (\text{gen})) \text{gen}$



the $[m+1]$ bracket puts the column back on the matrix, then the fold reads all of it

=



the fold reads the rest under F , then one gen puts the column in front

(13.5.2b)

$xs = 1 \ 2 \ 3 \ 4$

5 6 7 8

$: A[m][n], m=2$

$(\text{gen})(xs) = \text{gen}((1,2,3,4), (\text{gen})(5,6,7,8))$

$(\text{gen})(5,6,7,8)$

$= 5$

6

7

8

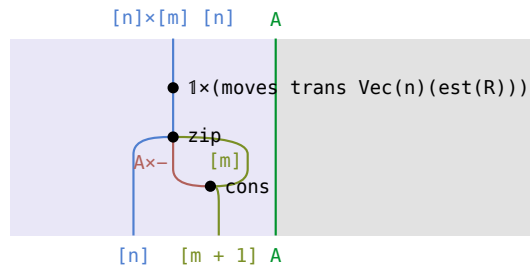
one column: each square its own one-square path

$: A[n][1][1]$

both halves are in (13.5.1b): the tail folds to the column 5 6 7 8, one path per row and each of them one square long, and gen on it is that walk.

§13.5.3. $Q \triangleq (1 \times (\text{moves trans Vec}(n)(\text{est}(R)))) \text{ zip}, A[n] \times A[n][m] \rightarrow A[n][m+1]$

(13.5.3a)



(13.5.3b)

$u = ((1,2,3,4), C)$
 $C = 5$
 6
 7
 8
 $(1 \times (\text{moves trans Vec}(n)(\text{est}(R))))$
 $\text{moves}(C)$
 $= 6 \ 7 \ 8 \ 5$
 $5 \ 6 \ 7 \ 8$
 $8 \ 5 \ 6 \ 7$
 $\text{trans}(\text{that})$
 $= 6 \ 5 \ 8$
 $7 \ 6 \ 5$
 $8 \ 7 \ 6$
 $5 \ 8 \ 7$
 $\text{Vec}(n)(\text{est}(R))(\text{that})$
 $= 5 \ 5 \ 6 \ 5$
 $\text{zip}(\text{that})$
 $= 1 \ 5$
 $2 \ 5$
 $3 \ 6$
 $4 \ 5$

$: A[n] \times A[n][m]$
 one path per row, one square long

 1 keeps the column, the paths move down, unmoved, up: a $3 \times n$ matrix

 $: A[3][n][m]$
 the transpose: row k gets rows $k-1, k, k+1$

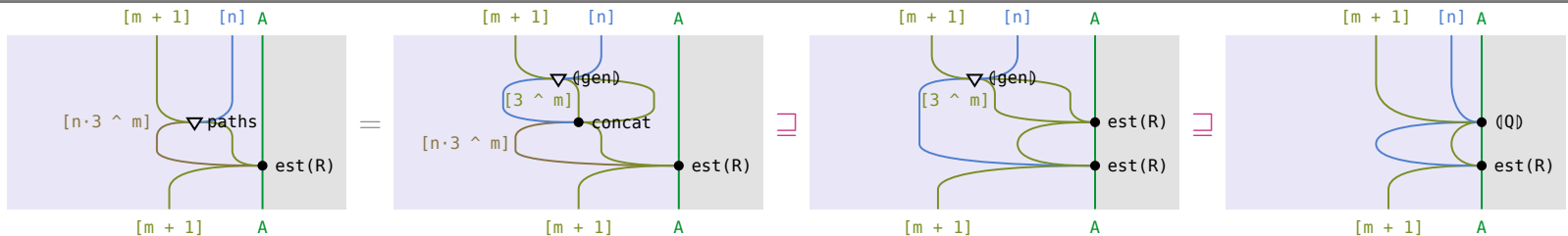
 $: A[n][3][m]$
 the cheapest into each row – chosen BEFORE
 $: A[n][m]$ the new square is put in front each row: its square, and the one predecessor that survived; the $[m+1]$ bracket is cons

 $: A[n][m+1]$

§13.5.4. paths $\text{est}(R) \sqsubseteq (Q) \text{est}(R)$

$(\forall i. 1 \in R) \wedge (\forall i. RR \in R) \wedge (\forall i. (1 \times R) \text{cons} \sqsubseteq \text{cons } R) \Rightarrow (Q) \text{est}(R) \sqsubseteq \text{paths est}(R)$ (13.5.4a)

a cheapest of all np paths of the cylinder is beaten by the greedy fold's one path per row and then a cheapest of those n , which costs $O(n \times m)$.



(13.5a) at paths

a cheapest of each of the n rows, then fusion at (13.5.4b)
 a cheapest of those; R transitive

$SS \sqsubseteq S \Rightarrow \text{Vec}(j)(\text{est}(S)) \text{est}(S) \sqsubseteq \text{concat est}(S)$, R transitive

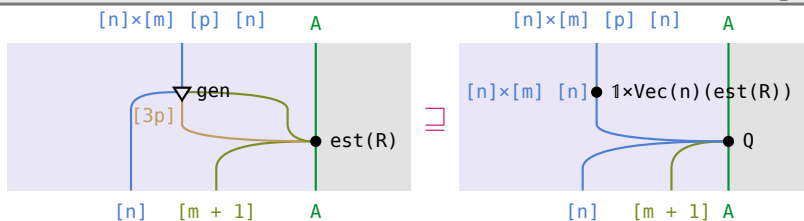
a cheapest of each of the j rows and then a cheapest of those j is a cheapest of all jk entries laid end to end.

$(\forall i. 1 \in R) \wedge (\forall i. RR \in R) \wedge (\forall i. (1 \times R) \text{cons} \sqsubseteq \text{cons } R) \Rightarrow (Q) \sqsubseteq (\text{gen}) \text{Vec}(n)(\text{est}(R))$, R reflexive, transitive and monotonic

the one path per row the greedy fold keeps is one of the p that (gen) generates for that row, and a cheapest of them.

$RR \in R \wedge (1 \times R) \text{cons} \sqsubseteq \text{cons } R \Rightarrow (1 \times \text{Vec}(n)(\text{est}(R))) \sqsubseteq \text{gen Vec}(n)(\text{est}(R))$ (13.5.4b)

choosing a cheapest of each square's p paths before the column is extended is no better than extending first and choosing among the $3p$.



(7.13), then zip, trans, moves
 lax natural

(7.13) on Vec: $(1 \times R) \text{cons} \sqsubseteq \text{cons } R \Rightarrow (1 \times \text{est}(R)) \text{cons} \sqsubseteq \text{cp Vec}(k)(\text{cons est}(R))$, R monotonic

putting the new square in front of every one of the p candidates and then choosing a cheapest is beaten by choosing a cheapest first and putting the square in front of that one.

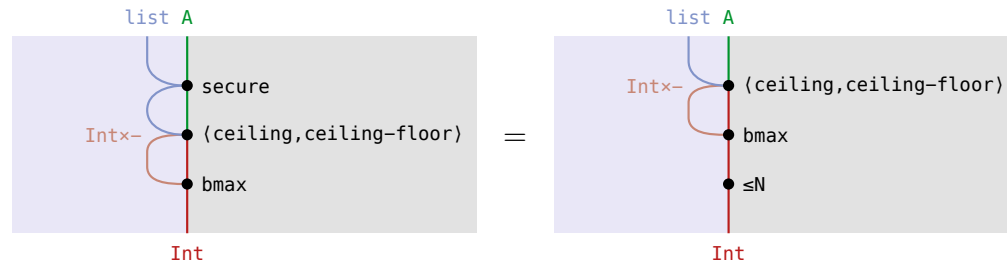
§13.6. The security van problem

(13.6a)

definition	type	example
$R \triangleq \text{length} \leq \text{length}^\circ$	$[[A]] \rightarrow [[A]]$	$[[a,b,c]]$ to $[[a],[b,c]]$, and not back: $1 \leq 2$.
$\text{ceiling} \triangleq \frac{\text{prefix sum}}{\ni} \text{est}(\geq)$	$[A] \rightarrow \text{Int}$	$\text{ceiling}[a,b]$ is the largest of $0, a, a+b$.
$\text{floor} \triangleq \frac{\text{prefix sum}}{\ni} \text{est}(\leq)$	$[A] \rightarrow \text{Int}$	$\text{floor}[a,b]$ is the smallest of $0, a, a+b$, so $[a,b]$ carries ceiling-floor cash.
secure the coreflexive on x with $\text{bmax}(\text{ceiling } x, \text{ceiling } x - \text{floor } x) \leq N$	$[A] \rightarrow [A]$	secure keeps $[a]$ exactly when $-N \leq a \leq N$.
ok the coreflexive on (a, xs) with xs non-empty and $[a] \# \text{head}(xs)$ secure	$A \times [[A]] \rightarrow A \times [[A]]$	ok keeps $(a, [[b],[c]])$ exactly when $[a,b]$ is secure.
$\text{new} \triangleq (\text{wrap} \times 1) \text{ cons}$	$A \times [[A]] \rightarrow [[A]]$	$\text{new}(a, [[b],[c]]) = [[a],[b],[c]]$.
$\text{glue} \triangleq (1 \times \text{cons}^\circ) \text{ assocl } (\text{cons} \times 1) \text{ cons}$	$A \times [[A]] \rightarrow [[A]]$	$\text{glue}(a, [[b],[c]]) = [[a,b],[c]]$.
$\text{old} \triangleq (1 \times \text{cons}^\circ) \text{ assocl } ((\text{cons secure}) \times 1) \text{ cons}$	$A \times [[A]] \rightarrow [[A]]$	$\text{old}(a, [[b],[c]]) = [[a,b],[c]]$ when $[a,b]$ is secure, and nothing otherwise.
$\text{partition} = ([\text{nil}, \text{new } u \text{ glue}])$ cutting the transactions into non-empty segments is one pass along them that either opens a segment for the transaction in hand or puts it on the front of the segment already open	$[A] \rightarrow [[A]]$	$\text{partition}[a,b]$ gives $[[a],[b]]$ and $[[a,b]]$.
$S \triangleq [\text{nil}, \text{new } u \text{ old}]$	$F[[A]] \rightarrow [[A]]$	$S(a, [[b],[c]])$ gives $[[a],[b],[c]]$, and $[[a,b],[c]]$ when $[a,b]$ is secure.
$(\forall a. \text{secureP}(\text{amount}, N, [a])) \Rightarrow$ $\text{partition list}(\text{secure}) = (S)$ cutting the transactions into non-empty segments every way and keeping the cuts whose every segment is secure is the one fold whose algebra is $[\text{nil}, \text{new } u \text{ old}]$	$[A] \rightarrow [[A]]$	Both take $[a,b]$ to $[[a],[b]]$ — each $[a]$ is secure — and to $[[a,b]]$ when $[a,b]$ is.
$H \triangleq (\text{head prefix}^\circ \text{ head}^\circ) u (\text{nil}^\circ \text{ nil})$	$[[A]] \rightarrow [[A]]$	$[[a],[b,c]]$ to $[[a,b],[c]]$: $[a]$ is a prefix of $[a,b]$.
$R; H \triangleq R n (R^\circ \Rightarrow H)$	$[[A]] \rightarrow [[A]]$	$[[a,b,c]]$ to $[[a],[b,c]]$ by $ R $, and $[[a],[b,c]]$ to $[[a,b],[c]]$ by $R n H$.
$ R \triangleq R n \neg R^\circ$	$[[A]] \rightarrow [[A]]$	$[[a,b,c]]$ to $[[a],[b,c]]$: $1 < 2$. The strict part R splits into: $R; H = R u (R n H)$.
the specification $\frac{\text{partition list}(\text{secure})}{\ni} \text{est}(R)$	$[A] \rightarrow [[A]]$	$[a,b]$ gives $[[a,b]]$ when $[a,b]$ is secure, and $[[a],[b]]$ otherwise.

§13.6.1. secure prefix $\Rightarrow$ prefix secure

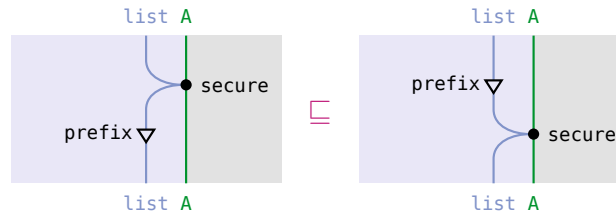
(13.6.1a)



$$\text{secure}(\text{ceiling}, \text{ceiling-floor}) \text{bmax} = (\text{ceiling}, \text{ceiling-floor}) \text{bmax} (\leq N)$$

a stretch passes **secure** before the test exactly where the test's own value passes $\leq N$ after it, which is $\triangleq$ **secure**'s **bmax**(**ceiling** **x**, **ceiling** **x** - **floor** **x**) $\leq N$

(13.6.1b)



$$\text{secure prefix} \sqsubseteq \text{prefix secure}$$

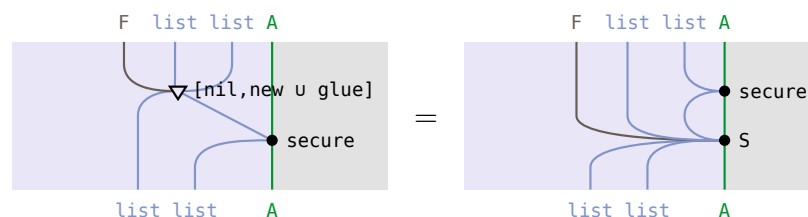
every pair **secure** then **prefix** gives, **prefix** then **secure** gives too — a prefix of a secure stretch is itself secure, which is the prefix-closure
B&dM p.185 names

§13.6.2. partition $\text{list}(\text{secure}) = ([\text{nil}, \text{new } u \text{ old}])$

(13.6.2a)

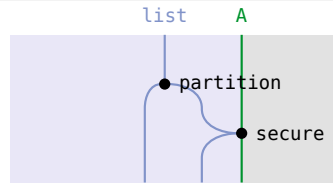
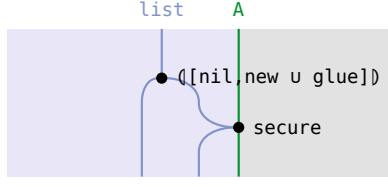
definition	Hinze–Marsden
$\text{new} \triangleq (\text{wrap} \times 1) \text{ cons}$ wrap makes the transaction a segment on its own, and cons puts that segment at the front of the schedule: the van is called	
$\text{glue} \triangleq (1 \times \text{cons}^\circ) \text{ assocl } (\text{cons} \times 1) \text{ cons}$ cons° splits the schedule into its first segment and the rest, the first cons puts the transaction at the front of that segment, and the second puts the segment back	
$\text{old} \triangleq (1 \times \text{cons}^\circ) \text{ assocl } ((\text{cons secure}) \times 1) \text{ cons}$ glue with secure on the segment the transaction has just joined: the van stays away only where the longer segment still passes	

(13.6.2b)

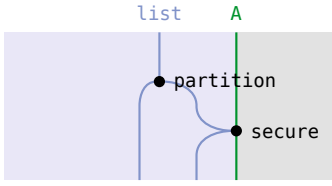
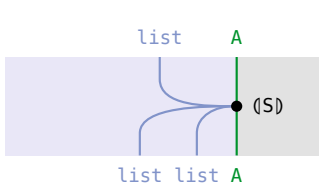


$$(\forall a. \text{secureP}(\text{amount}, N, [a])) \Rightarrow [\text{nil}, \text{new } u \text{ glue}] \text{list}(\text{secure}) = F(\text{list}(\text{secure})) S$$

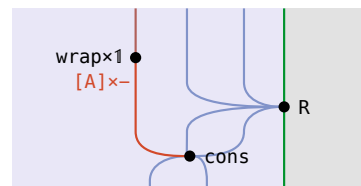
testing every segment for security after the algebra has run is testing it before it ran: a segment **glue** has extended is **secure** exactly where **old** would have let it be, and the one transaction **new** sets out on its own is **secure** whatever it is

$(\forall a. \text{secureP}(\text{amount}, N, [a])) \Rightarrow \text{partition list}(\text{secure}) = (S)$ (13.6.2c) cutting the transactions into non-empty segments every way and keeping the cuts whose every segment is secure is the one fold whose algebra is $[\text{nil}, \text{new } u \text{ old}]$	
step $\text{partition list}(\text{secure})$ the specification: cut the transactions every way, then keep the cuts whose every segment is secure	Hinze–Marsden 
$= ([\text{nil}, \text{new } u \text{ glue}]) \text{list}(\text{secure})$ partition is the fold that either opens a segment for the transaction in hand or puts it on the front of the segment already open	
$= ([\text{nil}, \text{new } u \text{ old}])$ fusion: the condition above moves $\text{list}(\text{secure})$ inside that fold, which is what turns glue into old	

(13.6.2d)

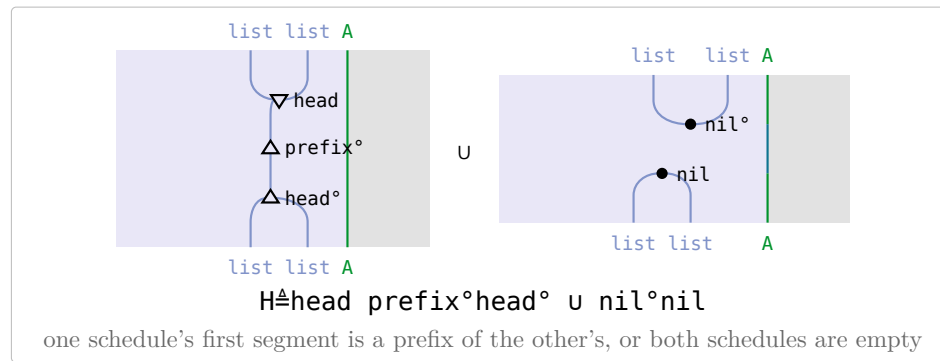
 	
$(\forall a. \text{secureP}(\text{amount}, N, [a])) \Rightarrow \text{partition list}(\text{secure}) = (S)$ cutting the transactions every way and then keeping the cuts whose every segment is secure is one pass along them that either calls the van or extends the open segment while it stays secure	

§13.6.3. $(1 \times R) \text{new} \sqsubseteq (\text{new } u \text{ old}) R$

$(1 \times R) \text{new} \sqsubseteq (\text{new } u \text{ old}) R$ (13.6.3a) calling the van for the transaction on a no-longer schedule gets no further than calling it on this one and shortening the schedule afterwards	
step $(1 \times R) \text{new}$ the left side of (7.14) $\triangleq \text{secure}$	Hinze–Marsden 
$= (\text{wrap} \times R) \text{cons}$ $\text{new} \triangleq (\text{wrap} \times 1) \text{cons}$ — (13.6.2a) — and $(\text{wrap} \times 1) (1 \times R) = (\text{wrap} \times R)$	
$\sqsubseteq \text{new } R$ cons is monotonic on R — $(1 \times R) \text{cons} \sqsubseteq \text{cons } R$, consing onto a no-longer schedule leaves it no longer — and $\text{new} \sqsubseteq \text{new } u \text{ old}$	

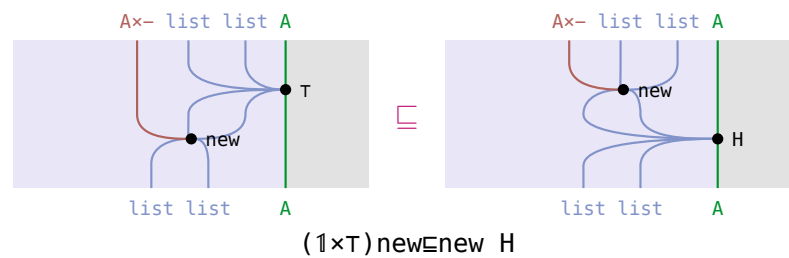
§13.6.4. $H \triangleq (\text{head prefix} \circ \text{head}) \cup (\text{nil} \circ \text{nil})$

(13.6.4a)



§13.6.5. $(1 \times (R;H)) \text{new} \in (\text{new} \cup \text{old})(R;H)$

(13.6.5a)



whatever schedule the van is called on, the result's first segment is the one transaction $[a]$, so any two results of **new** on that transaction stand in H

$(1 \times (R;H)) \text{new} \in (\text{new} \cup \text{old})(R;H)$

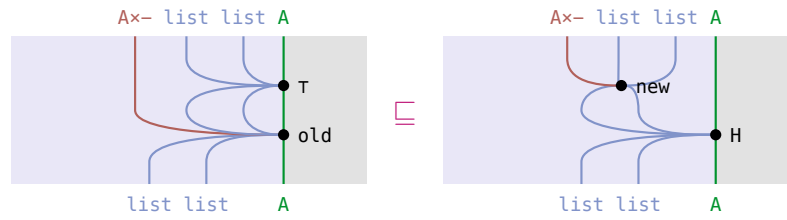
(13.6.5b)

calling the van for the transaction on a $R;H$ -better schedule gets no further than calling it on this one and bettering the whole schedule afterwards

step	Hinze–Marsden
$(1 \times (R;H)) \text{new}$ the left side of (7.16)	
$\sqsubseteq (1 \times R) \text{new}$ $R;H \in R \triangleq \text{secure}$	
$\sqsubseteq \text{new } R \cap \text{new } H$ (7.14) as far as its new R line — (13.6.3a) — and (7.18)	
$= \text{new } (R \cap H)$ new is a map, and a map distributes over $\cap$	
$\sqsubseteq \text{new } (R;H)$ $X \cap Y \in X;Y$, and $\text{new} \in \text{new} \cup \text{old}$	

§13.6.6. $(1 \times (R;H)) \text{old} \in (\text{new} \cup \text{old})(R;H)$

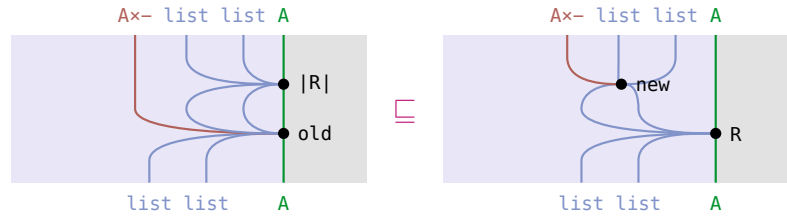
(13.6.6a)



$$(1 \times T) \text{old} \sqsubseteq \text{new H}$$

old leaves the transaction **[a]** at the front of the first segment, and **[a]** is what **new** makes that segment, so the two first segments are **prefix**-related whatever the schedules were

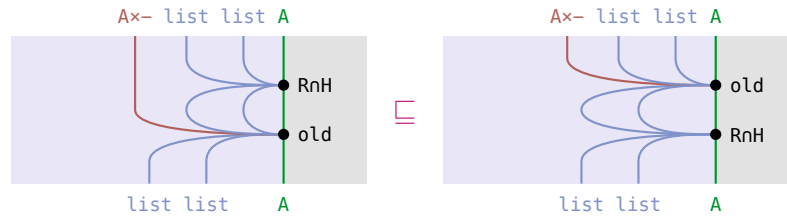
(13.6.6b)



$$(1 \times |R|) \text{old} \sqsubseteq \text{new R}$$

old keeps the schedule's length, so a strictly shorter one still comes out no longer than the one the van's own segment lengthens

(13.6.6c)



$$(1 \times (RnH)) \text{old} \sqsubseteq \text{old (RnH)}$$

on a tie the one first segment is a prefix of the other, so prefix-closure of **secure** — (13.6.1b) — lets **old** fire on this side too, and it keeps both the length and the prefix

$(1 \times (R;H)) \text{old} \sqsubseteq (\text{new} \cup \text{old})(R;H)$ (13.6.6d)	
gluing the transaction onto a better schedule for the rest gets no further than gluing it on, or calling the van, and bettering the whole schedule after,	
circuit — the old branch of each union	Hinze–Marsden
the old operand of new $\cup$ old , in every row	
$(1 \times R) \text{old} \cup (1 \times (R \cap H)) \text{old}$ $R;H = R \cup (R \cap H) \text{ — } \triangleq \text{secure, } U \text{ distributes,}$	
$\text{new } (R \cap H) \cup \text{old } (R \cap H)$ (7.19) and (7.20) on $ R $ — (13.6.6a), (13.6.6b) — and (7.21) on $R \cap H$ — (13.6.6c)	
$(\text{new} \cup \text{old})(R;H)$ $X \cap Y \sqsubseteq X;Y, \text{ converses}$	

§13.6.7. The derivation

$(\forall a. \text{secureP}(\text{amount}, N, [a])) \Rightarrow ([\text{nil}, (\text{ok} \rightarrow \text{glue}, \text{new})]) \sqsubseteq \frac{\text{partition list}(\text{secure})}{\supset} \text{est}(R)$ (13.6.7a)	
the fewest secure segments the transactions can be cut into are one pass along them, the next transaction glued onto the open segment wherever that segment stays secure and the van called where it does not	
$\frac{\text{partition list}(\text{secure})}{\supset} \text{est}(R)$ the specification — $\triangleq$ secure	$\frac{\text{partition list}(\text{secure})}{\supset} \text{est}(R)$ partition list(secure) = (S) — $\triangleq$ secure, fusion at secure prefix $\sqsubseteq$ prefix secure
$\frac{\text{partition list}(\text{secure})}{\supset} \text{est}(R;H)$ $R;H \sqsubseteq \text{secure};$ (7.15) $(1 \times R) \text{old} \sqsubseteq (\text{new} \cup \text{old})R$ is FALSE, the shorter partition need not stay secure, where (7.14) $(1 \times R) \text{new} \sqsubseteq (\text{new} \cup \text{old})R$ holds	$\frac{\text{partition list}(\text{secure})}{\supset} \text{est}(R;H)$ greedy theorem at $R;H$, its hypothesis $F(R;H) \sqsubseteq S(R;H)$ the old half (7.17) — (13.6.6d) — and the new half (7.16) — (13.6.5b) — which rests on (7.18) $(1 \times T) \text{new} \sqsubseteq \text{new } H$ — (13.6.5a)
$([\text{nil}, (\text{ok} \rightarrow \text{glue}, \text{new})]) \sqsubseteq \text{old} \sqsubseteq \text{new } (R;H) \circ: \text{old returns the shorter result wherever it returns one, and ok is where it does}$	